



NALMO: Transforming queries in natural language for moving objects databases

Xieyang Wang¹ · Mengyi Liu¹ · Jianqiu Xu¹ · Hua Lu²

Received: 20 January 2022 / Revised: 20 October 2022 / Accepted: 2 February 2023 /
Published online: 6 May 2023

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

Moving objects databases (MODs) have been extensively studied due to their wide variety of applications including traffic management, tourist service and mobile commerce. However, queries in natural languages are still not supported in MODs. Since most users are not familiar with structured query languages, it is essentially important to bridge the gap between natural languages and the underlying MODs system commands. Motivated by this, we design a natural language interface for moving objects, named NALMO. In general, we use semantic parsing in combination with a location knowledge base and domain-specific rules to interpret natural language queries. We design a corpus of moving objects queries for model training, which is later used to determine the query type. Extracted entities from parsing are mapped through deterministic rules to perform query composition. NALMO is able to well translate moving objects queries into structured (executable) languages. We support five kinds of queries including time interval queries, range queries, nearest neighbor queries, trajectory similarity queries and join queries. We develop the system in a prototype system SECONDO and evaluate our approach using 280 natural language queries extracted from popular conference and journal papers in the domain of moving objects. Four volunteers give the system satisfaction and related suggestions through three rounds of independent tests. Experimental results show that (i) *NALMO achieves accuracy and precision 96.8% and 81.1%, respectively*, (ii) *the average time cost of translating a query is 1.49s*, and (iii) *the average satisfaction is 95.5%*.

✉ Jianqiu Xu
jianqiu@nuaa.edu.cn

Xieyang Wang
xieyang@nuaa.edu.cn

Mengyi Liu
liumengyi@nuaa.edu.cn

Hua Lu
luhua@ruc.dk

¹ Nanjing University of Aeronautics and Astronautics, Nanjing, China

² Roskilde University, Roskilde, Denmark



NALMO: 将自然语言查询转换为移动对象数 据库查询

王谢阳¹ · 刘梦怡¹ · 徐建秋¹ · 卢华² 

接收日期: 2022年1月20日 / 修订日期: 2022年10月20日 / 接受日期: 2023年2月2日 / 在线发表日期: 2023年5月6日 © 作者, 专属许可授予施普林格科学商业媒体有限公司 (Springer Science Business Media, LLC), 施普林格自然 (Springer Nature) 的一部分 2023

摘要

移动对象数据库 (MODs) 因其广泛的应用而被广泛研究, 包括交通管理、旅游服务和移动商务。然而, 移动对象数据库仍不支持自然语言查询。由于大多数用户不熟悉结构化查询语言, 因此在自然语言与底层移动对象数据库系统命令之间架起桥梁至关重要。有鉴于此, 我们设计了一个面向移动对象的自然语言接口, 命名为NALMO。通常, 我们结合语义解析、位置知识库和特定领域规则来解释自然语言查询。我们为模型训练设计了一个移动对象查询语料库, 该语料库随后用于确定查询类型。从解析中提取的实体通过确定性规则映射以执行查询组成。NALMO能够将移动对象查询很好地翻译为结构化 (可执行) 语言。我们支持五种查询类型

✉ Jianqiu Xu
jianqiu@nuaa.edu.cn
Xieyang Wang
xieyang@nuaa.edu.cn
Mengyi Liu
liumengyi@nuaa.edu.cn
Hua Lu
luhua@ruc.dk

¹ Nanjing University of Aeronautics and Astronautics, Nanjing, China

² Roskilde University, Roskilde, Denmark

Keywords Moving objects database · Natural language interface · Semantic parsing · Structured language · Query processing

1 Introduction

Mobile computing devices such as PDAs and mobile phones have been widely used in real life. As a result, a large amount of mobile data is collected and stored in the system. Such data represents objects continuously changing locations over time. The system managing such data is called moving objects databases, MOD for short [17]. In the domain of moving objects databases, a number of queries and analysis tasks have been extensively studied including range queries [8], nearest neighbor queries [19], trajectory similarity queries and join queries. Several prototype systems are developed such as HERMES [37], SECONDO [18], PLACE* [54] and TMOM [25].

Despite extensive research having been studied in this area, there is lack of a query interface for non-expert users sending their requests in natural languages. It is a non-trivial task for most users to write structured languages such as SQL. Developing a natural language interface for databases (NLDBs) has become an urgent task as databases support an increasing number of applications. NLDBs will significantly benefit users as they can use natural languages to request a variety of queries instead of mastering the structured language as well as the underlying structure such as the database schema. Figure 1 depicts the task that we would like to achieve to empower MODs. That is, a natural language query is precisely and correctly translated into a structured and executable expression in MODs.

NLDB is a challenging task mainly due to two difficulties: (i) *semantic understanding and analysis*, and (ii) *query translation*. The questions asked by users are usually ambiguous or in-complete in terms of semantic. In the domain of moving objects databases a variety of queries have different key entities including time, location and moving objects identification. Constructing an effective semantic parser for identifying and extracting entities is a crucial task. Although MODs are already equipped with a rich set of operators and access methods,

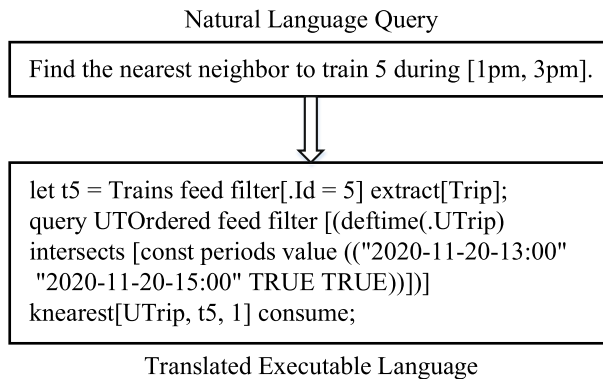


Fig. 1 Example of translating

关键词 移动物体数据库 · 自然语言界面 · 语义解析 · 结构化语言 · 查询处理

1 引言

移动计算设备如个人数字助理（PDA）和手机已经在现实生活中得到广泛使用。因此，大量的移动数据被收集并存储在系统中。这些数据表示随时间连续变化位置的对象。管理这些数据的系统称为移动对象数据库，简称MOD [17]。在移动对象数据库领域，许多查询和分析任务已经被广泛研究，包括范围查询 [8]、最近邻查询 [19]、轨迹相似性查询和连接查询。已经开发了几个原型系统，如HERMES [37]、SECONDO [18]、PLACE* [54] 和 TMOM [25]。

尽管该领域已经进行了大量研究，但对于非专业用户以自然语言发送请求仍缺乏查询接口。对于大多数用户来说，编写诸如 SQL 之类的结构化语言是一项不简单的任务。随着数据库支持的应用数量不断增加，开发数据库自然语言接口（NLDBs）已成为一项紧迫任务。NLDBs 将显著惠及用户，因为用户可以使用自然语言来请求各种查询，而无需掌握结构化语言以及如数据库模式等底层结构。图 1 描述了我们希望实现的任务，即赋能 MODs。这意味着，自然语言查询能够被精确且正确地翻译为 MODs 中可执行的结构化表达式。

自然语言接口数据库（NLIDB）是一项具有挑战性的任务，主要由于两个困难：(i) *semantic understanding and analysis*，以及 (ii) *query translation*。用户提出的问题通常在语义上是模糊的或不完整的。在移动对象数据库领域，各种查询包含不同的关键实体，包括时间、位置和移动对象标识。构建一个用于识别和提取实体的有效语义解析器是一项关键任务。尽管MODs已经配备了丰富的操作符和访问方法，

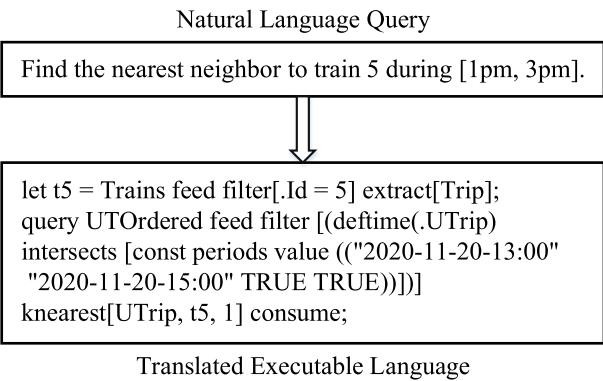


图 1 翻译示例

effectively and accurately combining entities and operators into structured languages is a non-trivial task due to the query diversity.

NLIDB systems have achieved satisfactory results in specific domains. Rule-based and machine learning-based methods are usually used to perform the translation from natural language query to SQL. Some NLIDB systems use machine learning [45] to understand and translate natural language queries such as *NL₂CM* [2] and *SpatialNLI* [28]. *PRECISE* [40], *NaLIR* [27] and *ATHENA* [43] use rule-based semantic analysis and query construction methods. These methods limit expressive power and can only be applied to specific domains. Natural language queries often contain ambiguities and grammatical errors which make mistakes and produce erroneous output. However, a purely rule-based method will greatly restrict the expression of queries that users can put forward. Natural language is inherently ambiguous, and helping NLIDB systems understand query semantics is challenging. All users can do is modify the query and re-query. Some NLIDBs ask users for intervention to perform query reformulation, such as *NaLIX* [31] and *NaLIR*. The users of NLIDB are usually not experts, and when the system gives users answer, they may have difficulty in judging whether it is correct. Some researchers have extended the capabilities of existing systems in terms of explaining both answer and non-answer to queries. *NLProv* [11] is able to provide natural language explanations for query answer. *NLProveNAns* [10] augments *NaLIR* by supporting interpretations of non-answer. State-of-the-art systems based on rule and machine learning methods have been able to translate natural language queries relatively well in different domains.

However, there is no NLIDB that supports MODs. Providing a user-friendly interface for non-experts from other domains is an important research direction of moving objects databases. An interactive interface is preferred, and users can check the correctness and accuracy of translated expressions [35]. Besides, most of the existing benchmarks are used for testing natural language queries of relational databases. For example, the ATIS [42] benchmark for air travel is based on a relational database called Official Airline Guide. The GeoQuery [46] benchmark for the geographic domain is based on the United States geographic database. The Scholar [20] benchmark consists of 816 natural language questions tagged with SQL for academic database search. Corpora that can be used for query classification of MODs are rare or even not available. There is no dataset for moving objects queries, and non-moving object queries cannot be used as test cases to determine the effect of the translation queries to MODs. Most current traditional database management systems including Oracle, MySQL, and ACCESS cannot handle time-varying, multi-dimensional, and variable-length data. The main reason is that spatial information including points, lines, and regions cannot be stored in the traditional two-dimensional table form. If we want to represent a moving object with spatial, temporal and attribute information and answer queries about its location, the location of the moving object needs to be updated continuously. For relational databases, it is impractical to store spatial data and update location information frequently. Compared with relational databases, MODs provide users with many basic operators (e.g., *filter*(●)) and special operators (e.g., *deftime*(●)) for different queries. Without these operators, natural language queries for moving objects are difficult to be effectively translated into corresponding relational database queries.

In this paper, we design and develop a natural language interface for MODs including (i) *semantic parsing and analysis*, (ii) *structured language composition*, and (iii) *selection of suitable datasets*. At the moment, the system supports five kinds of queries including time

由于查询的多样性，将实体和操作符有效且准确地组合成结构化语言是一项非平凡的任务。

NLIDB 系统在特定领域已取得令人满意的成果。通常使用基于规则和基于机器学习的方法将自然语言查询翻译为 SQL。一些 NLIDB 系统使用机器学习 [45] 来理解和翻译自然语言查询，例如 *NL₂CM* [2] 和 *SpatialNLI* [28]。PRECISE [40]、*NaLIR* [27] 和 *ATHENA* [43] 使用基于规则的语义分析和查询构建方法。这些方法限制了表达能力，只能应用于特定领域。自然语言查询通常包含歧义和语法错误，这可能导致错误并产生错误的输出。然而，纯粹的基于规则的方法会大大限制用户可以提出的查询表达。自然语言本质上具有歧义性，帮助 NLIDB 系统理解查询语义具有挑战性。用户唯一能做的就是修改查询并重新查询。一些 NLIDB 系统会要求用户干预以进行查询重构，例如 *NaLIX* [31] 和 *NaLIR*

然而，目前没有支持MODs的NLIDB。为来自其他领域的非专家提供用户友好的界面是动态图数据库的重要研究方向。更倾向于交互式界面，用户可以检查翻译表达的正确性和准确性 [35]。此外，大多数现有的基准测试用于测试关系数据库的自然语言查询。例如，用于航空旅行的ATIS [42] 基准测试是基于名为Official Airline Guide的关系数据库。地理领域的GeoQuery [46] 基准测试是基于美国地理数据库。Scholar [20] 基准测试包括816个带有SQL标签的自然语言问题，用于学术数据库搜索。可用于MODs查询分类的语料库非常罕见甚至不可用。目前没有用于移动对象查询的数据集，非移动对象查询也不能作为测试用例来确定翻译查询的效果。

●)) 和特殊运算符 (例如, *deftime*(●)) 对于不同的查询来说。如果没有这些运算符，关于移动对象的自然语言查询很难被有效地转换为对应的关系数据库查询。

在本文中，我们为 MODs 设计和开发了一种自然语言接口，包括 (i) *semantic parsing and analysis*、(ii) *structured language composition* 和 (iii) *selection of suitable datasets*。目前，该系统支持五种查询类型，包括时间

interval queries, range queries, nearest neighbor queries, trajectory similarity queries and join queries, but not limited to the five types. This is because the corresponding operators and functions are implemented. Compared with [51], we further support join query. The corpus and test dataset of join query are also constructed. If the operators and functions of other queries and analysis are well handled in the system, those queries and analysis can be well supported. To support natural language queries, the first issue is semantic understanding. We preprocess user-specified spatio-temporal queries in which there may exist punctuation that can affect the processing result or be unnecessary. These punctuations should be replaced with spaces, before subsequent work can be carried out. After preprocessing, word segmentation and entity recognition are performed by appropriate natural language processing tools. The extraction of key semantic information is one of the most important parts of natural language translation. We propose a location knowledge base and a domain methodology for entity analysis such that a word like “*Nanjing*” can be parsed as a location. We extract the city’s POI information through OpenStreetMap (OSM) and allow users to supplement POI information to improve the location knowledge base [51].

We collect more than 60 conference and journal papers to extract query examples, and manually generate moving objects queries for knowledge enhancement to build a moving objects query corpus. The model trained through the corpus can accurately determine the query type. We further refine the scoring criteria of experts for moving object queries [51]. After the type determination, query interpretations are mapped into query fragments based on inference rules, before finally being assembled into complete queries. We implement our proposal in the system SECONDO that provides users with many basic operators and special operators. According to different query types, the parsed entities are mapped to corresponding query fragments and these fragments are combined with operators to obtain the final structured query. Efficiency is also a significant part of the user experience. In order to improve the translation efficiency of the system, we use a location-based prefix index in the process of extracting location entities. It is worth noting that most of the query objects in our examples are public transportation, because one of our test datasets is moving underground trains. However, we can still query moving objects excluding public transportation as long as there are corresponding datasets. We conduct a separate experimental test of the quality of natural language understanding, and ask four volunteers to participate in the user interaction tests. Finally we expound system development, especially how to integrate NALMO into SECONDO [51]. Three criteria are defined to evaluate the translation quality: (i) *accuracy and precision*, (ii) *time cost* and (iii) *usability*.

The main contributions of this paper are summarized as follows.

- We design an algorithm for spatio-temporal data parsing in which a location knowledge base is utilized to extract location information. We describe the methods to expand the location knowledge base. A location-based prefix index is developed to improve the extraction efficiency.
- We propose a corpus for query type determination and develop methods for constructing structured query languages which combine the parsed key entities and system operators. We show detailed scoring rules for query extraction, including conference, journal papers and expert scoring rules.
- We develop NALMO in an extensible database system SECONDO and conduct experimental evaluation using two datasets. We introduce how to integrate NALMO in

区间查询、范围查询、最近邻查询、轨迹相似性查询和连接查询，但不限于这五种类型。这是因为相应的操作符和函数已经实现。与[51]相比，我们进一步支持连接查询。连接查询的语料库和测试数据集也已构建。如果系统能够很好地处理其他查询和分析的操作符和函数，那么这些查询和分析也可以得到良好的支持。为了支持自然语言查询，第一个问题是语义理解。我们预处理用户指定的时空查询，其中可能存在影响处理结果或不必要的标点符号。这些标点符号应替换为空格，然后才能进行后续工作。预处理之后，将通过适当的自然语言处理工具进行分词和实体识别。关键词义信息的提取是

我们收集了60多篇会议和期刊论文以提取查询示例，并手动生成移动对象查询以增强知识，从而构建移动对象查询语料库。通过该语料库训练的模型可以准确地确定查询类型。我们进一步完善了专家对移动对象查询的评分标准[51]。在类型确定之后，查询解释会根据推理规则映射为查询片段，然后最终组装成完整查询。我们在系统SECONDO中实现了我们的方案，该系统为用户提供了许多基本操作符和特殊操作符。根据不同的查询类型，解析出的实体会映射到相应的查询片段，这些片段再与操作符结合以获得最终的结构化查询。效率也是用户体验的重要组成部分。为了提高系统的翻译效率，我们在提取过程中使用基于位置的前缀索引。

本文的主要贡献总结如下。

- 我们设计了一种时空数据解析算法，其中利用位置知识库来提取位置信息。我们描述了扩展位置知识库的方法。开发了基于位置的前缀索引以提高提取效率。
- 我们提出了一个用于查询类型确定的语料库，并开发了用于构建结构化查询语言的方法，这些方法结合了解析出的关键实体和系统操作符。我们展示了查询提取的详细评分规则，包括会议、期刊论文和专家评分规则。
- 我们在一个可扩展的数据库系统 SECONDO 中开发 NALMO，并使用两个数据集进行实验评估。我们介绍如何将 NALMO 集成到

SECONDO. The experiment results demonstrate that our method achieves accuracy and precision of 96.8% and 81.1%, respectively. The average time cost of translating a query is 1.49s. Through the three rounds of independent testing conducted by four volunteers, the user satisfaction is 95.5%.

The rest of the paper is organized as follows. We review the related work in Section 2 and provide the framework of NALMO in Section 3. We introduce natural language understanding and the query translation in Sections 4 and 5, respectively. The experimental evaluation is presented in Section 6. We introduce how to integrate NALMO in SECONDO in Section 7, followed by the conclusion in Section 8.

2 Related work

MODs have been extensively studied in a range of applications including transportation system, express delivery services and mobile commerce [48, 58]. In the literature, a number of techniques are proposed such as data models [16, 21, 55], index structures [6, 38, 47, 49], query algorithms [8, 19] and algorithms of location selection problems [32]. However, there is still lack of a natural language interface in moving objects databases which supports transforming natural language expressions into executable languages.

2.1 Challenges of NLDBs

Natural Language Interfaces (NLI) have a long history in the field of natural language processing and the database community, and the implementation of NLDB has also received extensive attention from researchers. NLDB does not require users to master any underlying knowledge and query language of the database, helping users to realize the function of querying the database through natural language description. Unsurprisingly, the advent of the big data era has made NLDB more attractive than ever. However, despite years of research, NLDB remains largely an open research problem, with no general commercial solutions or prototype systems as yet. Two major open challenges hinder the wide adaption of NLDBs: natural language understanding and query translation [29].

2.1.1 Natural language understanding

To have good NLDBs, we need to understand natural language problems well. A number of systems use rule-based methods to understand natural language in a specific domain. These systems are usually restricted in natural language support such as grammar and vocabulary to improve the accuracy of semantic understanding [56]. *PRECISE* [40] defines the notion of semantic tractability and identifies a subset of natural language queries that can be precisely translated into SQL. *NaLIX* [31] limits natural language queries to a controlled subset based on a predefined grammar. To understand the problem of mixing individual and general knowledge, *NL₂CM* [2] uses the IX detector, implemented through declarative selection patterns combined with specialized vocabularies, to distinguish individual query parts from general ones. An ideal NLDB enables users to make arbitrarily complex temporary queries on the underlying database and obtain accurate information with the least effort. Therefore, in the

SECONDO。实验结果表明，我们的方法分别达到了96.8%的准确率和81.1%的精确率。翻译一次查询的平均时间成本为1.49秒。通过四位志愿者进行的三轮独立测试，用户满意度为95.5%。

论文的其余部分组织如下。我们在第二节回顾相关工作，并在第三节提供NALMO的框架。我们分别在第四节和第五节介绍自然语言理解和查询翻译。第六节展示实验评估。第七节介绍如何在SECONDO中集成NALMO，第八节是结论。

2 相关工作

MODs 在包括交通系统、快递服务和移动商务等一系列应用中已经得到广泛研究 [48, 58]。在文献中，提出了许多技术，如数据模型 [16, 21, 55]、索引结构 [6, 38, 47, 49]、查询算法 [8, 19] 以及位置选择问题的算法 [32]。然而，在移动对象数据库中，仍然缺乏支持将自然语言表达转换为可执行语言的自然语言接口。

2.1 自然语言接口数据库的挑战

自然语言接口（NLI）在自然语言处理和数据库领域有着悠久的历史，而NLIDB的实现也获得了研究人员的广泛关注。NLIDB不要求用户掌握任何底层知识和数据库查询语言，帮助用户通过自然语言描述实现数据库查询功能。不出所料，大数据时代的到来使NLIDB比以往任何时候都更具吸引力。然而，尽管进行了多年的研究，NLIDB在很大程度上仍然是一个未解决的研究问题，目前还没有通用的商业解决方案或原型系统。两个主要的未解决挑战阻碍了NLIDB的广泛应用：自然语言理解和查询翻译 [29]。

2.1.1 自然语言理解

要拥有良好的自然语言接口数据库系统（NLIDBs），我们需要很好地理解自然语言问题。一些系统使用基于规则的方法来理解特定领域的自然语言。这些系统通常在自然语言支持方面受到限制，例如语法和词汇，以提高语义理解的准确性 [56]。*PRECISE* [40] 定义了语义可处理性的概念，并识别出一部分可以精确转换为 SQL 的自然语言查询。*NaLIX* [31] 将自然语言查询限制在基于预定义语法的受控子集内。为了理解混合个体知识和通用知识的问题，*NL₂CM* [2] 使用 IX 检测器，该检测器通过声明性选择模式结合专门词汇实现，以区分个体查询部分和通用部分。理想的 NLIDB 允许用户在底层数据库上进行任意复杂的临时查询，并以最小的努力获得准确的信息。因此，在

process of semantic understanding, *NALIX* and *DiaSQL* [15] modify the query in subsequent user interactions to reformulate the parse tree, but the reformulation usually needs excessive user interactions. *NaLIR* [27] detects the parse tree so that users can directly modify the parse tree instead of the questions raised by the user. It can generate suggestions for users to replan queries when they are outside the semantic scope. *ATHENA* [43] constructs a translation index through specific entity extraction, and uses ontology-based domain knowledge to bridge the semantic gaps caused by leaky abstraction. *ATHENA++* [43, 62] further translates complex nested SQL queries. *NL2TRANQUYL* [5] stores and models relevant information by employing ontology which contains different concepts. *QUERIX* [61] analyzes the syntax of natural language by a syntactic analyzer, which requires that the natural language components must be complete. Otherwise, the results obtained by the syntax analyzer are not accurate enough, which may decrease the accuracy of final results.

With the increasing use of statistical learning techniques, attempts have been made to perform semantic analysis on sentences in the form of various supervision. Pasupat and Liang [36] use question and answer pairs as supervision to solve the answering task of complex questions on semi-structured tables. Poon and Domingos [39] is the first attempt at learning a semantic parsing model in an unsupervised manner [24]. Artzi and Zettlemoyer [3] learn what the user's utterances represent by obtaining feedback from the conversation. Wang et al. [52] successfully realize the training of a semantic parser in a domain without training examples. Their framework consists of two parts: a builder and a domain general grammar. Wong and Mooney [53] use statistical machine translation technology to complete semantic parsing tasks.

In recent years, neural networks [20] are increasingly used to train models of parsing natural language sentences, encoding and decoding for natural language semantic understanding. Although frameworks based on encoder-decoder frameworks offer greater flexibility, they generally do not allow interpretation and understanding of meaning combinations [24]. The approach adopted by Cheng et al. [7] builds the intermediate structure within two stages, allowing for a very clear view of what the model is learning. Similar to this, an intermediate template is also generated in [12], where the final output is sketchy for easier decoding. Yin and Neubig [57] alleviate the need for models to discover inherent syntax from scarce training data by explicitly providing constraints for decoders in the form of target language syntax. The work of the model can then be better focused on building parsing, guided by the known grammar rules.

2.1.2 Query translation

On the basis of a positive understanding of natural language, how to translate natural language is also of crucial importance. The common method of query construction is to use a unique algorithm to construct a structured query from the meaning of natural language query and the domain knowledge corresponding to the underlying database [30]. *ATHENA* proposes a unique two-stage approach that first translates an input natural language query into an ontology query language (OQL) query defined over the ontology, and then translates the OQL query into its corresponding SQL query. *NLPQC* [44] accepts queries based on specific domain templates. *QUERIX* extracts specific components from the syntax tree to match knowledge with the knowledge base to obtain the final result. Up-to-date machine learning technologies are gradually applied to natural language translation. *NL₂CM* [2] mines

语义理解的过程, *NALIX* 和 *DiaSQL* [15] 在后续用户交互中修改查询以重新构建解析树, 但这种重构通常需要过多的用户交互。*NaLIR* [27] 检测解析树, 使用户可以直接修改解析树, 而不是修改用户提出的问题。当查询超出语义范围时, 它可以生成建议帮助用户重新规划查询。*ATHENA* [43] 通过特定实体提取构建翻译索引, 并利用基于本体的领域知识弥合由于抽象不完整造成的语义差距。*ATHENA++* [43, 62] 进一步翻译复杂的嵌套 SQL 查询。*NL2TRANQUYL* [5] 通过使用包含不同概念的本体来存储和建模相关信息。*QUERIX* [61] 通过语法分析器分析自然语言的语法, 这要求自然语言的组成部分必须完整, 否则语法分析器得到的结果将不够准确, 这 m

随着统计学习技术的日益广泛应用, 人们尝试以各种监督形式对句子进行语义分析。*Pasupat* 和 *Liang* [36] 使用问答对作为监督来解决半结构化表上复杂问题的回答任务。*Poon* 和 *Domingos* [39] 是首次尝试以无监督方式学习语义解析模型 [24]。*Artzi* 和 *Zettlemoyer* [3] 通过对对话中获取反馈来学习用户的表达所代表的内容。*Wang* 等 [52] 成功实现了在没有训练样本的领域中训练语义解析器。他们的框架由两个部分组成: 一个构建器和一个领域通用语法。*Wong* 和 *Mooney* [53] 使用统计机器翻译技术来完成语义解析任务。

近年来, 神经网络 [20] 被越来越多地用于训练自然语言句子的解析模型, 以及自然语言语义理解的编码和解码。尽管基于编码器-解码器框架的框架提供了更大的灵活性, 但它们通常不允许对意义组合进行解释和理解 [24]。*Cheng* 等人采用的方法 [7] 在两个阶段内构建中间结构, 从而可以非常清晰地看到模型在学习什么。类似地, [12] 中也生成了一个中间模板, 其中最终输出是草图式的, 以便于解码。*Yin* 和 *Neubig* [57] 通过以目标语言语法的形式明确提供约束, 减轻了模型从稀缺训练数据中发现固有语法的需求。模型的工作随后可以更好地集中在构建解析上, 并由已知语法规则指导。

2.1.2 查询翻译

在对自然语言有积极理解的基础上, 如何翻译自然语言也具有至关重要的意义。常见的查询构建方法是使用独特的算法, 根据自然语言查询的意义和底层数据库对应的领域知识来构建结构化查询 [30]。*ATHENA* 提出了一种独特的两阶段方法, 首先将输入的自然语言查询翻译为在本体上定义的本体查询语言 (OQL) 查询, 然后再将 OQL 查询翻译为其对应的 SQL 查询。*NLPQC* [44] 基于特定领域模板接受查询。*QUERIX* 从语法树中提取特定组件, 以将知识与知识库匹配, 从而获得最终结果。最新的机器学习技术正在逐步应用于自然语言翻译。*NL₂CM* [2] 挖掘

the crowd for individual by translating questions of the audience into OASSIS-QL which is defined as an extension of SPARQL. *SpatialNLI* [28] uses seq2seq translation to enhance its translation model. A number of systems combine deterministic algorithm approaches and machine learning methods to construct structured languages [33]. A parser generates corresponding SQL clauses, then further constructs SQL query candidates according to the corresponding templates and rules, and finally ranks the candidates of the constructed languages through the SVM ranker [14].

2.2 User interaction in NLIDBs

Based on the technical approach they use, NLIs can be divided into four categories: keyword-, pattern-, parsing- and grammar-based NLI [1]. User interaction is involved in each type. *QUICK* [59] is a keyword-based NLI, which refines user interaction on the basis of keyword search to enhance the expressiveness of semantic queries. In concrete implementation, after the user raises a keyword question, the system guides the user to select the intention of the question through progressive refinement steps. *NLQ/A* [60] is a pattern-based NLI to query a knowledge graph [23]. The system optimizes the user interaction part to better resolve the ambiguity problem. As a parsing-based NLI, *NaLIR* is an improved version of *NaLIX*. It not only explains to the user how to process the query, but also shows a variety of understandings for the user to choose from to ease the user's burden of eliminating misunderstandings. In grammar-based systems, take *SPARKLIS* [13] as an example. To ensure the completeness of user input when searching for concepts, entities or modifiers, the system uses a cascade of three stages.

These NLIDBs that contain user interaction provide guidance and security to users through intermediate answer and suggestions. When the user inputs a query question, the user is guided to input the correct query statement by demonstrating some available query statements and prompting the types of statements that can be queried. When the question raised by the user is semantically ambiguous, the correct semantic information is determined by providing the user with possible explanations to choose from. When the information entered by the user cannot be retrieved in the database, the system can retrieve whether the user has entered similar information existing in the database, and provide an association prompt for incorrect input, thereby helping the user to complete the query correctly. However, too many interactions and constraints not only reduce the translation speed of the system, but also reduce the user experience. Considering that the main purpose of user interaction is to improve system effectiveness, researchers have gradually begun to use existing data to improve system effectiveness. For example, *TEMPLAR* [4] uses SQL query log information to improve the ability of existing NLIDB to perform keyword mapping and join path inference, greatly improving the accuracy.

2.3 NLIDBs for spatio-temporal databases

NLIDB for spatio-temporal databases has also received attention in recent years [9]. *TEQUILA* [22] proposes a knowledge graph to deal with temporal questions, where cues for temporal relations need to be discovered and handled. Wang et al. propose a spatial understanding model *SpatialNLI* based on contextual semantics [28], which can recognize

通过将观众的问题翻译成 OASSIS-QL（被定义为 SPARQL 的扩展），为个体服务的群体。*SpatialNLI* [28] 使用 seq2seq 翻译来增强其翻译模型。一些系统结合确定性算法方法和机器学习方法来构建结构化语言 [33]。解析器生成对应的 SQL 子句，然后根据相应的模板和规则进一步构建 SQL 查询候选，最后通过 SVM 排序器 [14] 对构建语言的候选进行排序。

2.2 自然语言数据库接口中的用户交互

根据所使用的技术方法，自然语言接口（NLI）可以分为四类：基于关键字、基于模式、基于解析和基于语法的 NLI [1]。每种类型都涉及用户交互。*QUICK* [59] 是一种基于关键字的 NLI，它在关键字搜索的基础上优化用户交互，以增强语义查询的表达能力。在具体实现中，当用户提出关键字问题后，系统通过逐步改进步骤引导用户选择问题的意图。*NLQA* [60] 是一种用于查询知识图谱 [23] 的基于模式的 NLI。系统优化了用户交互部分，以更好地解决歧义问题。作为基于解析的 NLI，*NaLIR* 是 *NaLIX* 的改进版本。它不仅向用户解释如何处理查询，还展示了多种理解供用户选择，以减轻用户消除误解的负担。在基于语法的系统中，以 *SPARKLIS* [13] 为例。为了确保用户的完整性

这些包含用户交互的自然语言接口数据库（NLIDBs）通过中间答案和建议为用户提供指导和安全保障。当用户输入查询问题时，通过展示一些可用的查询语句并提示可查询的语句类型，引导用户输入正确的查询语句。当用户提出的问题从语义上存在歧义时，通过提供可能的解释供用户选择来确定正确的语义信息。当用户输入的信息在数据库中无法检索时，系统可以检索用户是否输入了数据库中已存在的相似信息，并为错误输入提供关联提示，从而帮助用户正确完成查询。然而，过多的交互和约束不仅会降低系统的翻译速度，还会降低用户体验。考虑到用户交互的主要目的是提高

2.3 用于时空数据库的自然语言界面数据库系统

近年来，面向时空数据库的自然语言接口查询（NLIDB）也受到了关注 [9]。*TEQUILA* [22] 提出了一个知识图谱来处理时间问题，其中需要发现和处理时间关系的线索。王等人提出了一个基于上下文语义的空间理解模型 *SpatialNLI* [28]，该模型可以识别

the meaning of spatial entities. Spatial terms are extracted from natural language description through symbolic expressions to represent spatial features and relations between them [41]. *NL2TRANQUYL* [5] is a system primarily for trip-planning requests in a large multimodal transportation system with different constraints such as taking the least time, traveling the shortest distance and choosing the least expensive route.

NALMO is designed for queries over moving objects. Some NLDBs [5, 27] use different parsers to obtain dependency parser tree for a given query, and the quality of parser tree depends largely on the parser. *ATHENA* uses the TIMEX annotator to find all time ranges involved in NLQ, the Stanford Numeric Expressions annotator to find all tokens containing numeric quantities, and the Stanford Dependency Parser to identify dependencies between dependants and dependees. *NaLIR* uses Stanford Parser [26] to generate language parsing trees from natural language queries. For MODs, the location and time information is very critical. We do not rely much on natural language processing tools and will further process natural language queries to extract key entities. Majority of the existing NLDBs construct queries via query composition. These NLDBs all map the processed key information to the corresponding structured components. However, for various queries, different DBMSs and structured languages provide diverse operators and clauses, requiring different mapping values. *ATHENA* uses the mapping between the ontology and the database schema to translate the intermediate query language on the ontology into SQL. *NaLIR* needs to determine whether the query tree contains function nodes or quantifier nodes, and then construct it according to the situation. NALMO considers multiple query types in moving objects and designs different mapping templates for each query type, while others mainly use parse tree nodes and ontology elements for mapping information.

Most natural language interfaces of semantic web rely on the use of general dictionaries for resource matching, and WordNet [34] is also used for dictionary expansion. However, general dictionaries are far from meeting the needs of users in certain domains, and thus, specific domain dictionaries should be added. What is more, the lack of benchmark and datasets of NLDBs also catches much attention. There are already a few datasets such as MAS [27] in *NaLIR* which is suitable for academic papers, authors, conferences and universities. GEO [40] in *PRECISE* is for spatial queries but without time information. None of those works deal with natural language queries in moving objects databases. NALMO proposes a dataset containing multiple moving object queries.

3 The framework

We provide an overview of NALMO in Fig. 2. Key components are (i) a *semantic parser*, (ii) a *location knowledge base*, (iii) a *location-based prefix index*, (iv) a *corpus for query*

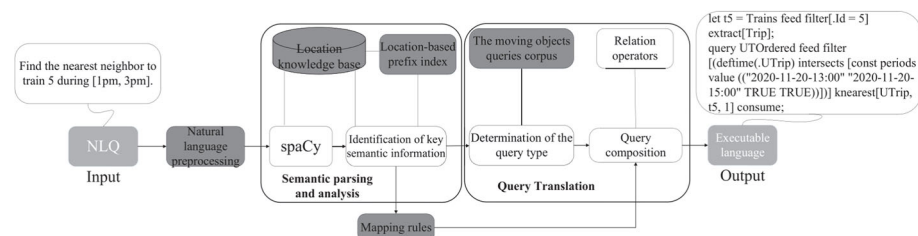


Fig. 2 The NALMO architecture in SECONDO

空间实体的意义。空间术语通过符号表达从自然语言描述中提取，以表示空间特征及其之间的关系 [41]。NL2TRANQUYL [5] 是一个主要用于大型多模式交通系统中旅行规划请求的系统，该系统具有不同的约束条件，例如用时最短、行程最短和选择最便宜的路线。

NALMO 是为移动对象查询而设计的。一些自然语言数据库接口 (NLIDBs) [5, 27] 使用不同的解析器来获取给定查询的依存解析树，而解析树的质量在很大程度上取决于解析器。ATHENA 使用 TIMEX 注释器来找到 NLQ 中涉及的所有时间范围，使用 Stanford 数字表达式注释器来找到包含数字量的所有词元，并使用 Stanford 依存解析器来识别依赖项与被依赖项之间的依赖关系。NaLIR 使用 Stanford Parser [26] 从自然语言查询生成语言解析树。对于移动对象数据库 (MODs)，位置信息和时间信息非常关键。我们不太依赖自然语言处理工具，将进一步处理自然语言查询以提取关键实体。现有的大多数 NLIDBs 通过查询组合构建查询。这些 NLIDBs 都将处理后的关键信息映射到相应的结构化组件。然而，对于各种查询，不同的数据库管理系统 (DBMS) 和结构化语言提供

语义网的大多数自然语言接口依赖于使用通用词典进行资源匹配，并且 WordNet [34] 也用于词典扩展。然而，通用词典远不能满足特定领域用户的需求，因此应添加特定领域词典。更重要的是，自然语言数据库接口 (NLIDB) 缺乏基准和数据集也引起了广泛关注。已经存在一些数据集，例如 NaLIR 中的 MAS [27]，适用于学术论文、作者、会议和大学。PRECISE 中的 GEO [40] 用于空间查询，但不包含时间信息。这些工作都没有涉及移动对象数据库中的自然语言查询。NALMO 提出了一个包含多个移动对象查询的数据集。

3 框架

我们在图2中提供了NALMO的概述。关键组件包括(i) *a semantic parser*, (ii) *a location knowledge base*, (iii) *a location-based prefix index*, (iv) *a corpus for query*

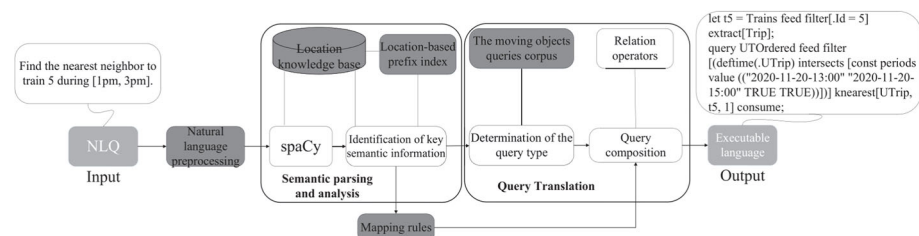


图 2 SECONDO 中的 NALMO 架构

type determination and (v) *a rule-based algorithm for structured language generation*. At first, the module of natural language processing performs semantic analysis and extracts key entities from the query expression. Next, the query type is determined by a machine learning model and there are five kinds of queries in total, summarized in Table 1. Finally, NALMO maps the entities to queries and combines moving objects operators with entities to produce structured queries. Frequently used notations are summarized in Table 2. Using the query Q5 in Table 1, we illustrate the workflow as follows.

Semantic interpreter We use a semantic parser to identify and extract key entities. We preprocess the punctuation of natural language and leverage the tool *spaCy* for word segmentation and entity recognition. The *spaCy* parser enables NALMO to realize that token numbers (e.g., five and 3) and time (e.g., 8am and 10am) are attached to labels CARDINAL and TIME, respectively. Then, we develop a rule-based algorithm to further extract the location, time (e.g., 8am and 10am), the number of nearest neighbors (e.g., five) and the moving objects identifier (e.g., 3) from parsed entities.

Location knowledge base and location-based prefix index In addition to moving objects, the database also stores relevant point and region objects representing interesting places, e.g., POIs. We find out those objects and put them into the location knowledge base that serves as the dictionary. If the query location does not exist in the dictionary, the location information misses and users will not get the expected results. As location description usually contains a large number of prefixes, we use prefix indices to increase the efficiency of matching between natural language queries and the location knowledge base.

Query type determination We build a corpus for moving object queries in natural languages and utilize an LSTM neural network to train a model for predicting the query type. For example, in Table 1 the expression Q5 “Which trains are the nine continuous nearest neighbors to the train 9 from 8am to 10am.” will be reported as a nearest neighbor query.

Structured language generation NALMO maps the identified key information to the corresponding module in moving objects queries and combines them with defined operators

Table 1 Query examples and types

Id	Natural Language Query Example	Type
Q1	Where did the trains go at a certain time t_1 ?	Time Interval Query
Q2	Do you know where did exactly the train O_1 go during the time period $[t_2, t_3]$?	Time Interval Query
Q3	Please tell me the time that the train 7 arrive at Nanjing Station?	Range Query
Q4	Find the trains that pass the station Nantong during the time interval $[t_4, t_5]$?	Range Query
Q5	Which trains are the nine continuous nearest neighbors to the train 9 from 8am to 10am.	Nearest Neighbor Query
Q6	Show me k nearest neighbors to the train O_2 at any time instance of the time period $[t_6, t_7]$.	Nearest Neighbor Query
Q7	Tell me the trajectory that is similar to the train 7 at any time instance of the time period $[t_8, t_9]$.	Trajectory Similarity Query
Q8	Find the same trajectory between train O_3 and train O_4 during the time period $[t_{10}, t_{11}]$.	Join Query

type determination 以及 (v) *a rule-based algorithm for structured language generation*。首先，自然语言处理模块执行语义分析并从查询表达式中提取关键实体。接下来，通过机器学习模型确定查询类型，总共有五种查询类型，汇总在表 1 中。最后，NALMO 将实体映射到查询，并将移动对象运算符与实体结合以生成结构化查询。常用符号汇总在表 2 中。使用表 1 中的查询 Q5，我们演示工作流程如下。

语义解释器 我们使用语义解析器来识别和提取关键实体。我们对自然语言的标点进行预处理，并利用工具 *spaCy* 进行分词和实体识别。*spaCy* 解析器使 NALMO 能够识别出标记数字（例如，five 和 3）和时间（例如，8am 和 10am）分别对应于 CAR DINAL 和 TIME 标签。然后，我们开发了一种基于规则的算法，从解析出的实体中进一步提取位置、时间（例如，8am 和 10am）、最近邻数量（例如，five）以及移动对象标识符（例如，3）。

位置知识库和基于位置的前缀索引 除了移动对象外，数据库还存储表示感兴趣地点的相关点和区域对象，例如兴趣点（POI）。我们会找出这些对象并将其放入作为字典的位置知识库中。如果查询位置不存在于字典中，则位置信息缺失，用户将无法获得预期结果。由于位置描述通常包含大量前缀，我们使用前缀索引来提高自然语言查询与位置知识库之间匹配的效率。

查询类型确定 我们构建了一个用于自然语言运动对象查询的语料库，并使用 LSTM 神经网络来训练一个模型以预测查询类型。例如，在表 1 中，表达式 Q5 “Which trains are the nine continuous nearest neighbors to the train 9 from 8am to 10am.” 将被报告为最近邻查询。

结构化语言生成 NALMO 将识别出的关键信息映射到移动对象查询中相应的模块，并将其与定义的操作符结合起来

表 1 查询示例和类型

Id	Natural Language Query Example	Type
Q1	Where did the trains go at a certain time t_1 ?	Time Interval Query
Q2	Do you know where did exactly the train O_1 go during the time period $[t_2, t_3]$?	Time Interval Query
Q3	Please tell me the time that the train 7 arrive at Nanjing Station?	Range Query
Q4	Find the trains that pass the station Nantong during the time interval $[t_4, t_5]$?	Range Query
Q5	Which trains are the nine continuous nearest neighbors to the train 9 from 8am to 10am.	Nearest Neighbor Query
Q6	Show me k nearest neighbors to the train O_2 at any time instance of the time period $[t_6, t_7]$.	Nearest Neighbor Query
Q7	Tell me the trajectory that is similar to the train 7 at any time instance of the time period $[t_8, t_9]$.	Trajectory Similarity Query
Q8	Find the same trajectory between train O_3 and train O_4 during the time period $[t_{10}, t_{11}]$.	Join Query

Table 2 Frequently used notations

Name	Abbreviation
A natural language interface for moving objects	NALMO
Natural language interfaces to database	NLIDB
Moving objects database	MOD
Moving objects queries	MOQ
Berlin trains	BT

to construct structured queries. The expression $Q5$ is a nearest neighbor query, and thus corresponding operators (e.g., $nearestneighbor(\bullet)$) are used to form the expression.

4 Natural language understanding

The task is to precisely capture the meaning of a natural language expression and accurately translate the expression into the query language. Database queries have domain specific features or key attributes, which help analyze the semantics. Effectively extracting those information is essential in understanding natural languages. The procedure is as follows. We retain or delete the punctuations, and utilize the tool *spaCy* to perform word segmentation and do entity recognition.

4.1 Natural language processing

NALMO deletes character interference items such as double quotation marks (“ ”), single quotation marks (‘ ’), exclamation (!) and the right square bracket (]). The left square bracket ([) should be kept because the symbol guarantees the correct identification of time.

After the deletion, we use the industrial-grade *spaCy* parser for word segmentation and entity recognition. Time symbols are labelled TIME, and number symbols are labelled CARDINAL. These time and number symbols are stored for further identification. Numbers may be in the form of English and must be converted to Arabic numbers. If a number is combined with a connector, we replace the connector with a space. For numbers with space such as “one hundred”, we use the following regular expression to cut the space and identify each word.

$$re = re.split('[]', n) \quad (1)$$

After removing the conjunction word “and” in numbers, we traverse the remaining English digits in reverse order to obtain the digits such as ten, hundred and thousand. These numbers are eventually converted to Arabic numbers.

4.2 Identifying key semantic information

Since entities are approximately processed by *spaCy*, key information for moving objects queries is not accurate and the results are not sufficient for structured language construction. To increase the translation quality, we propose an algorithm to further parse the spatio-temporal data in Algorithm 1.

表 2 常用符号

Name	Abbreviation
A natural language interface for moving objects	NALMO
Natural language interfaces to database	NLIDB
Moving objects database	MOD
Moving objects queries	MOQ
Berlin trains	BT

构建结构化查询。表达式 $Q5$ 是一个最近邻查询，因此相应的操作符（例如， $nearestneighbor(\bullet)$ ）用于形成表达式。

4 自然语言理解

任务是精确捕捉自然语言表达的含义，并将该表达准确地翻译为查询语言。数据库查询具有特定领域的特征或关键属性，这有助于分析语义。有效地提取这些信息对于理解自然语言至关重要。流程如下：我们保留或删除标点符号，并使用工具 *spaCy* 进行分词和实体识别。

4.1 自然语言处理

NALMO 删除字符干扰项，例如双引号（“ ”）、单引号（‘ ’）、感叹号（!）和右方括号（]）。左方括号（[）应该保留，因为该符号可以确保时间的正确识别。

删除后，我们使用工业级的 *spaCy* 解析器进行分词和实体识别。时间符号被标注为 TIME，数字符号被标注为 CARDINAL。这些时间和数字符号会被存储以便进一步识别。数字可能以英文形式出现，必须转换为阿拉伯数字。如果数字与连接符结合，我们将连接符替换为空格。对于带空格的数字，例如 “one hundred”，我们使用以下正则表达式去掉空格并识别每个词。

$$re = re.split('[]', n) \quad (1)$$

在去掉数字中的连词 “and” 后，我们将剩余的英文数字倒序遍历，以得到十、百、千等数字。这些数字最终会被转换为阿拉伯数字。

4.2 识别关键语义信息

由于实体是由 *spaCy* 大致处理的，移动对象查询的关键信息不准确，结果不足以进行结构化语言构建。为了提高翻译质量，我们提出了一种算法，以进一步解析算法 1 中的时空数据。

Algorithm 1 Semantic parsing.

Input: query expression, Q ;
prefix tree of location knowledge base, $TrieLDB$;
relation knowledge base in database, RKB ;

Output: the parsing result, Q'

```

1:  $doc \leftarrow nlp(Q)$ 
2:  $tmp\_token \leftarrow find(doc, "neighbor")$ 
3: for  $token_i \in doc$  do
4:   if  $token_i.label\_ = CARDINAL$  then
5:      $N\_list.append(token_i)$ 
6:   end if
7: end for
8: for  $token_i \in doc$  do
9:   if  $token_i.label\_ = TIME$  then
10:     $Q'.time \leftarrow token_i$ 
11:   end if
12:   if  $token_i.pos\_ = NOUN \vee token_i.pos\_ = PROP$  then
13:     if  $search(TrieLDB, token_i.text)$  then
14:        $Q'.location \leftarrow token_i$ 
15:     end if
16:     if  $token_i.text \in RKB$  then
17:        $Q'.relation \leftarrow token_i$ 
18:       if  $token_{i+1} \in N\_list$  then
19:          $Q'.object\_id = token_{i+1}$ 
20:       end if
21:     end if
22:   end if
23:   if  $token_i = tmp\_token$  then
24:      $Q'.number\_NN \leftarrow token_{i-1}$ 
25:   end if
26: end for

```

Given a time word with colon such as “8:00 o'clock”, we use the following expression for matching and extraction.

$$r' \backslash d + [::] \backslash d + ' \quad (2)$$

The contents in “[:]” are optional characters. We take both Chinese and English colons into account as users may type them in a mixed way. A Chinese colon may appear in an English sentence, causing the time extraction error. If the time in the query includes a Chinese colon, the colon is replaced by an English one. For the hour without a colon such as “8am”, we use the following expression for matching and extraction.

$$r' \backslash d + ' \quad (3)$$

Using a regular expression for matching a full hour is simple, but the result cannot be used in the database. The full hour needs to be attached with “:00”. That is, “8am” will be rewritten as “8:00” and the terms “am” and “pm” will be converted to 24-hour format. We traverse the word segmentation results and evaluate each segmented word using the location knowledge base to determine whether the query location exists or not. Query relations are extracted from the database. We analyze the objects in the database and find the relation containing an attribute *mpoint*. The data type *mpoint* is for representing moving objects. This relation has target moving objects. There may be an atomic flow containing only one object with the attribute *mpoint* or a relation that may have multiple moving objects, and the latter is required. An example is provided in Fig. 3 in which there is a relation with the schema *Trains*(*Id:int*, *Trip:mpoint*). *Trains*() is suitable for time interval query and range

Algorithm 1 Semantic parsing.

Input: query expression, Q ;
 prefix tree of location knowledge base, $TrieLDB$;
 relation knowledge base in database, RKB ;
Output: the parsing result, Q'

```

1:  $doc \leftarrow nlp(Q)$ 
2:  $tmp\_token \leftarrow find(doc, "neighbor")$ 
3: for  $token_i \in doc$  do
4:   if  $token_i.label\_ = CARDINAL$  then
5:      $N\_list.append(token_i)$ 
6:   end if
7: end for
8: for  $token_i \in doc$  do
9:   if  $token_i.label\_ = TIME$  then
10:     $Q'.time \leftarrow token_i$ 
11:   end if
12:   if  $token_i.pos\_ = NOUN \vee token_i.pos\_ = PROP$  then
13:     if  $search(TrieLDB, token_i.text)$  then
14:        $Q'.location \leftarrow token_i$ 
15:     end if
16:     if  $token_i.text \in RKB$  then
17:        $Q'.relation \leftarrow token_i$ 
18:       if  $token_{i+1} \in N\_list$  then
19:          $Q'.object\_id = token_{i+1}$ 
20:       end if
21:     end if
22:   end if
23:   if  $token_i = tmp\_token$  then
24:      $Q'.number\_NN \leftarrow token_{i-1}$ 
25:   end if
26: end for
    
```

对于带有冒号的时间词，例如 “8:00 o’clock”，我们使用以下表达式进行匹配和提取。

$$r'\backslash d + [::]\backslash d + ' \quad (2)$$

“[]” 中的内容是可选字符。我们会考虑中英文冒号，因为用户可能会混合输入它们。中文冒号可能出现在英文句子中，导致时间提取错误。如果查询中的时间包含中文冒号，则将其替换为英文冒号。对于没有冒号的小时数，例如 “8am”，我们使用以下表达式进行匹配和提取。

$$r'\backslash d + ' \quad (3)$$

使用正则表达式匹配整点很简单，但结果不能直接用于数据库。整点需要附加上 “.00”。也就是说，“8am” 将被重写为 “8:00”，而 “am” 和 “pm” 将被转换为 24 小时格式。我们遍历分词结果，并使用位置知识库评估每个分词，以确定查询位置是否存在。从数据库中提取查询关系。我们分析数据库中的对象，并找到包含属性 *mpoint* 的关系。数据类型 *mpoint* 用于表示运动对象。该关系有目标运动对象。可能存在仅包含具有属性 *mpoint* 的一个对象的原子流，或者可能有多个运动对象的关系，而后者是所需的。图 3 提供了一个示例，其中有一个关系，其模式为 *Trains(Id:int, Trip:mpoint)*。 *Trains()* 适合时间区间查询和范围。

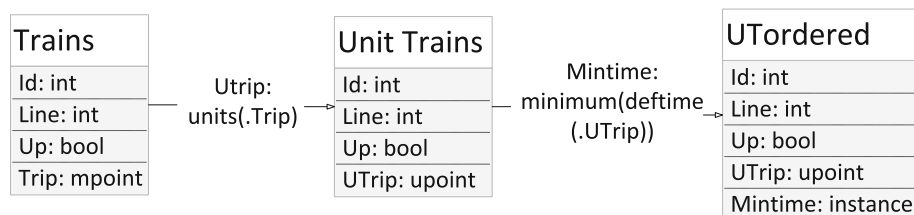


Fig. 3 Example of extracted relations in Berlin Trains dataset

query. Based on the extracted relation, we can extend the existing relation for different query types. *UnitTrains()* adds a new unit attribute *UTrip* on the basis of *Trains()*. *UTordered()* is sorted by start time on the basis of *UnitTrains()*. The two relations are applied to nearest neighbor query and similarity query. We use the relations in Berlin Trains dataset as examples, but not limited to these relations.

The semantics of numbers may be ambiguous in some cases as the number could be the identification of a query object or the number of neighbors. For example, in *Q5*, the moving objects relation in the database is *train*, and the number 9 exists in the number list. Thus, 9 will be an object identifier. In an English query, the number of neighbors must appear before the word “neighbor”, or the user may only query the nearest neighbor without the number of neighbors in the sentence. We utilize this knowledge to determine the semantics.

The analysis of time complexity Assuming that the number of words contained in a natural language query is n , the length of a location string is l , and the number of relations in the database is m , it can be seen that the time complexity of finding location by prefix tree is $O(l)$ (line 13), and the time complexity of determining the query relation is $O(m)$ (line 16). If l is greater than m , the time complexity of Algorithm 1 is $O(n \cdot l)$. Otherwise, the time complexity is $O(n \cdot m)$.

4.3 Location knowledge base and index

Spatial information needs to be captured in moving objects queries. In order to determine the locations, we generate a location knowledge base for each database which contains all the locations included in the database. The location knowledge base extracts objects whose attribute is *point* or *region*, because in *SECONDO* only objects whose attribute is *point* or *region* can be locations. If the location does not exist in the location knowledge base, we generate corresponding structured languages, but users will not get the expected results.

Location information is important for the construction of structured queries of moving objects, and it is necessary to capture spatial information in moving object queries. In order to determine the required location information, the system can automatically generate a location knowledge base for each database, which contains all possible locations in the database. When the dataset is modified or replaced, the system can still generate the corresponding location knowledge base for the database. The test dataset is Berlin train timetable dataset and the following location extractions are based on Berlin and other cities in Germany. According to the three attributes of POI information name, latitude and longitude, and POI description, the POI information collection of Berlin is extracted from OSM. At the same time, we carry out POI information extraction for other cities in Germany. When the required dataset is selected, the location knowledge base will automatically determine the city of the dataset and add the corresponding city POI information to the dataset. If the information already exists, we can

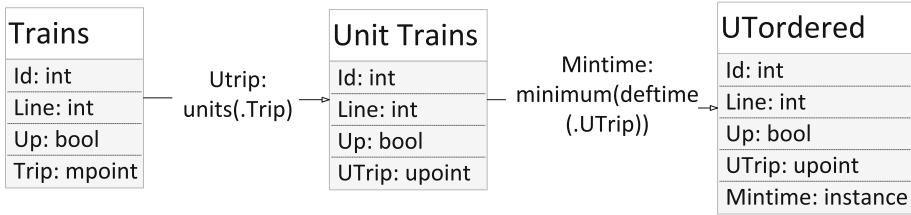


图 3 柏林火车数据集中提取关系的示例

查询。基于提取的关系，我们可以针对不同的查询类型扩展现有关系。*UnitTrains()* 在 *Trains()* 的基础上添加了一个新的单元属性 *UTrip*。*UOrdered()* 在 *UnitTrains()* 的基础上按开始时间排序。这两个关系应用于最近邻查询和相似性查询。我们以柏林火车数据集中的关系为例，但不限于这些关系。

在某些情况下，数字的语义可能是模糊的，因为数字可能是查询对象的标识符或邻居的数量。例如，在 *Q5* 中，数据库中的移动对象关系是 *train*，数字列表中存在数字 9。因此，9 将是一个对象标识符。在英语查询中，邻居的数量必须出现在单词 “neighbor” 之前，否则用户可能只会查询最近的邻居而句子中没有邻居数量。我们利用这些知识来确定语义。

时间复杂度分析 假设自然语言查询中包含的单词数为 n ，位置字符串的长度为 l ，数据库中的关系数为 m ，可以看出，通过前缀树查找位置的时间复杂度为 $O(l)$ (行13)，确定查询关系的时间复杂度为 $O(m)$ (行16)。如果 l 大于 m ，算法1的时间复杂度为 $O(n \cdot l)$ 。否则，时间复杂度为 $O(n \cdot m)$ 。

4.3 位置知识库与索引

在移动对象查询中，需要捕获空间信息。为了确定位置，我们为每个数据库生成一个位置知识库，其中包含数据库中包含的所有位置。位置知识库会提取属性为 *point* 或 *region* 的对象，因为在 *SECONDO* 中，只有属性为 *point* 或 *region* 的对象才能是位置。如果位置不存在于位置知识库中，我们会生成相应的结构化语言，但用户将无法获得预期的结果。

位置信息对于移动对象的结构化查询构建非常重要，在移动对象查询中有必要捕获空间信息。为了确定所需的位置信息，系统可以为每个数据库自动生成一个位置知识库，该知识库包含数据库中所有可能的位置。当数据集被修改或替换时，系统仍然可以为数据库生成相应的位置信息知识库。测试数据集是柏林火车时刻表数据集，以下位置提取基于柏林及德国其他城市。根据POI信息的三个属性：名称、经纬度和POI描述，从OSM中提取柏林的POI信息集合。同时，我们对德国其他城市进行POI信息提取。当选择所需的数据集时，位置知识库将自动确定城市。

ignore the location. The location information in the location knowledge base mainly contains the following three sources: (i) objects with attributes of point, line and region existing in the dataset; (ii) POI information of the corresponding city extracted by OSM; (iii) user queried location information which does not exist in the constructed location knowledge base and the existing POI collection of other cities. User can give feedback and the administrator can manually add the location information.

In order to improve the query efficiency, we construct a location-based prefix index for matching the location knowledge base. A large number of locations often contain the same prefix. Therefore, the index can effectively improve the extraction efficiency of location information. For example, “Nanjing” (Q3) and “Nantong” (Q4) have the same prefix “Nan”. The root node of the prefix tree is empty, and every child node contains an English character. From the root node to a certain leaf node, the characters passing on the path are connected, which is the location corresponding to the nodes. The characters contained in all child nodes of each node are different from each other. Characters that repeat continuously from the first character occupy only one node. Besides, we merge some nodes with only one child node as the leaf nodes to reduce memory consumption. In the process of prefix tree construction, whenever there is a location, we will start matching from the root node. If there is no matching node for the character, we will create a new node and continue to match the next character. If the node already exists, we follow the path to continue matching. An example of a location-based prefix tree is shown in Fig. 4. With such prefix trees, locations can be determined efficiently.

5 Query translation

The translation consists of two steps: (i) *determining the query type* and (ii) *constructing the structured language*. Semantic understanding and parsing of natural languages produce key entities and map them to structured query languages. There are different types of queries and a corpus is built to determine the query type. The corpus makes use of an LSTM neural network to train a model for identifying the query type. The natural language query needs to be proposed according to a certain grammar, and the moving objects query must contain the key information. Therefore, the corpus is more important for the result of query type determination, and the choices of neural network classifiers have less influence. NALMO selects the mapping method and combines the affluent operators to form structured queries.

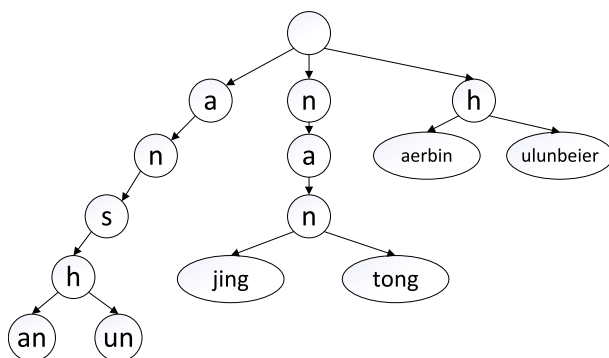


Fig. 4 Example of location-based prefix tree

忽略位置。位置知识库中的位置信息主要包含以下三个来源：(i) 数据集中存在的具有点、线和区域属性的对象；(ii) 通过 OSM 提取的对应城市的 POI 信息；(iii) 用户查询的、在已构建的位置知识库和其他城市现有 POI 集合中不存在的位置信息。用户可以提供反馈，管理员可以手动添加位置信息。

为了提高查询效率，我们构建了一个基于位置的前缀索引来匹配位置知识库。大量的位置通常包含相同的前缀。因此，该索引可以有效地提高位置信息的提取效率。例如，“*Nanjing*” (Q3) 和 “*Nantong*” (Q4) 有相同的前缀 “*Nan*”。前缀树的根节点为空，每个子节点包含一个英文字母。从根节点到某个叶子节点，路径上的字符连接起来，即为对应节点的地点。每个节点的所有子节点中包含的字符彼此不同。连续重复的字符仅占一个节点。此外，我们将一些只有一个子节点的节点合并为叶子节点，以减少内存消耗。在前缀树构建过程中，无论何时出现某个位置，我们都会从根节点开始匹配。如果没有与该字符匹配的节点

5 查询翻译

翻译分为两个步骤：(i) *determining the query type* 和 (ii) *constructing the structured language*。自然语言的语义理解和解析产生关键实体，并将其映射到结构化查询语言。查询类型有多种，并构建了语料库以确定查询类型。该语料库利用 LSTM 神经网络训练模型以识别查询类型。自然语言查询需要根据特定语法提出，移动对象查询必须包含关键信息。因此，语料库对于查询类型确定的结果更为重要，而神经网络分类器的选择影响较小。NALMO 选择映射方法，并结合丰富的操作符生成结构化查询。

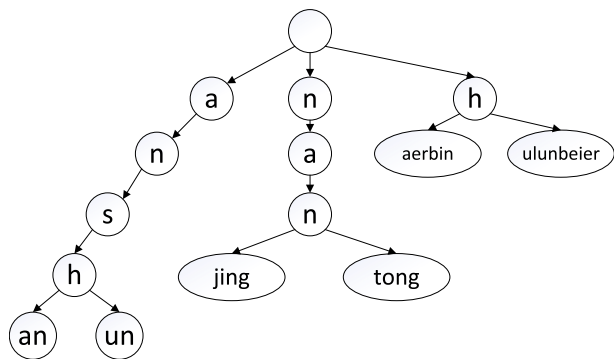


图 4 基于位置的前缀树示例

5.1 Corpus: determining query type

Providing a common entity mapping and query combination method is a challenging task because moving object queries contain specific key information which require different operators. One solution of determining the query type is to use defined rules, but the strategy will produce a set of fixed outputs. When the natural language expression changes, the method probably produces incorrect results. The determination effect primarily depends on the quality of natural language processing tools.

To have high quality processing, we build a moving object corpus. The corpus contains 1500 moving objects queries classified into five categories including (i) time interval query, (ii) range query (iii) nearest neighbor query, (iv) trajectory similarity query, (v) join query. It is worth noting that time interval query includes time point query. Range query includes spatial query and spatio-temporal range query. In the process of classification and training, we also consider the special cases of time interval query and range query. We initially collect 350 moving objects query examples from more than 60 published conference and journal papers, and then use experts' knowledge, entity information replacement and sentence expression replacement to enhance the knowledge. Experts utilize various key information with different semantic expressions according to the query type. These queries satisfy the diversity of semantic expression properly, and the corpus is extended up to 1500 moving objects queries. The experts participating in our work specialize in database, specifically MODs. We manually mark each query.

Data extraction is guided by two criteria: (i) *knowledge of moving objects experts* and (ii) *citation-based impact factors from the Institute for Scientific Information (ISI)*. We use popular academic conferences and journals as references, including 34 journal papers and 28 conference papers. Experts judge the quality of natural language queries in articles and rank these queries. ISI calculates the impact factor of the journal based on its citation index. The assumption is that articles in higher-quality journals and conferences are likely to provide comprehensive queries. Each journal and conference receives a ranking score. The highest ranking journal or conference receives a score 1, and the next highest ranking receives a score 1/2. The ranking score is used as a reference, and the final score is given by the expert. Articles published in the same journal have the same scores. Domain experts rank query examples according to the query robustness, and the maximum score is 1. Then, the article score and the expert score are summarized as the total score, and moving objects queries ranking from high to low will be added in the corpus. Journal and conference scores are given in Table 3.

Experts will also score the different query examples. The scoring criteria of experts are as follows (the query results are accurate to two decimal places). Experts should determine whether the query is semantically complete and whether there are obvious grammatical errors. If the query semantics is incomplete or there are grammatical errors, the score will be 0, and the subsequent judgment will be terminated. Experts judge whether the query belongs to the natural language query of the moving object. If not, the score will be directly 0, and the subsequent judgment is terminated. This module accounts for 0.2 points. Experts judge whether the query is a moving object query type supported by the system. As long as the query belongs to any type of moving objects query, the query gets the score. If the expert cannot determine, they can give scores as appropriate. The query type that is not supported by the system will directly be 0, and the subsequent judgment will be terminated. This module accounts for 0.4 points. Experts judge whether the query contains the entity required by the corresponding query type. If the required entity is missing or the entity expression is ambiguous, the score of the query will be deducted. This module accounts for 0.4 points. An example of expert scoring is shown in Table 4.

5.1 语料库：确定查询类型

提供一种通用的实体映射和查询组合方法是一项具有挑战性的任务，因为移动对象查询包含特定的关键信息，需要不同的操作符。确定查询类型的一种解决方案是使用定义的规则，但该策略会产生一组固定的输出。当自然语言表达发生变化时，该方法可能会产生错误的结果。确定效果主要取决于自然语言处理工具的质量。

为了实现高质量的处理，我们构建了一个移动对象语料库。该语料库包含1500个移动对象查询，分类为五类，包括：(i) 时间间隔查询，(ii) 范围查询，(iii) 最近邻查询，(iv) 轨迹相似性查询，(v) 连接查询。值得注意的是，时间间隔查询包括时间点查询。范围查询包括空间查询和时空范围查询。在分类和训练过程中，我们还考虑了时间间隔查询和范围查询的特殊情况。我们最初从60多篇已发表的会议和期刊论文中收集了350个移动对象查询示例，然后利用专家知识、实体信息替换和句子表达替换来增强知识。专家根据查询类型利用各种具有不同语义表达的关键信息。这些查询恰当地满足了语义表达的多样性，语料库扩展到1500个移动对象。

数据提取由两个标准指导：(i) *knowledge of moving objects experts* 和 (ii) *citation-based impact factors from the Institute for Scientific Information (ISI)*。我们以热门的学术会议和期刊作为参考，包括34篇期刊论文和28篇会议论文。专家评判文章中自然语言查询的质量，并对这些查询进行排序。ISI 根据期刊的引用指数计算期刊影响因子。假设高质量期刊和会议中的文章可能提供全面的查询。每个期刊和会议都会收到一个排名分数。排名最高的期刊或会议得分为1，次高排名得分为1/2。排名分数作为参考，最终评分由专家给出。同一期刊发表的文章得分相同。领域专家根据查询的稳健性对查询示例进行排名，最高分为1。随后，将文章分数和专家分数汇总为总分，并对移动物体查询进行排序

专家还将对不同的查询示例进行评分。专家的评分标准如下（查询结果精确到小数点后两位）。专家应判断查询的语义是否完整，以及是否存在明显的语法错误。如果查询语义不完整或存在语法错误，则得分为0，后续判断将终止。专家判断查询是否属于移动对象的自然语言查询。如果不是，得分将直接为0，后续判断终止。该模块占0.2分。专家判断查询是否为系统支持的移动对象查询类型。只要查询属于任何一种移动对象查询，该查询即可得分。如果专家无法判断，可以酌情给分。不被系统支持的查询类型将直接为0分，后续判断将终止。该模块

Table 3 Scores of journals and conferences

Journal and Conference Proceeding	Score
ACM Conference on Management of Data	1
IEEE Transactions on Knowledge and Data Engineering	1
Proceedings of the VLDB Endowment	1
IEEE International Conference on Data Engineering	1
The VLDB Journal	1
GeoInformatica	1/2
International Conference on Extending DB Technology	1/2
ACM SIGSPATIAL	1/2
Journal of Computer Science and Technology	1/2
Journal of Database Management	1/3
International Symposium on Spatial and Temporal Databases	1/3
International Conference on Mobile Data Management	1/3

When determining the specific model for the corpus training, the LSTM network is chosen based on the following considerations:

(i) In the classification problem, although feedforward networks represented by CNN still have performance advantages, the LSTM can realistically represent or simulate the cognitive process of human behavior, logical development and neural organization, and can handle complex tasks.

(ii) The number of queries in the corpus is not particularly large, and an overly complex deep learning model can not be selected. If a complex model with a large number of parameters is used for training, it is likely to underfit the model. Compared with the Transformer and Bert models proposed recently, the LSTM is simpler and easier to implement.

Table 4 Expert scoring examples

Natural Language Query Examples	Moving Objects Query	Query Type Support	Entity Integrity	Total Score
Where were the trains at a certain time t1?	0.2	0.4	0.4	1
How many trains at Nanjing station and show me the closest way to the station?	0.2	0.15	0	0.35
Is there any one see the red car?	0	0	0	0
What trains pass the station Nantong during [8am, 9qm]?	0.2	0.2	0.4	1
Show me five nearest neighbors to the train 3 right now.	0.2	0.4	0.25	0.85
When did the train pass underground Nanjing Station?	0.2	0.4	0.3	0.9

表 3 期刊和会议的评分

Journal and Conference Proceeding	Score
ACM Conference on Management of Data	1
IEEE Transactions on Knowledge and Data Engineering	1
Proceedings of the VLDB Endowment	1
IEEE International Conference on Data Engineering	1
The VLDB Journal	1
GeoInformatica	1/2
International Conference on Extending DB Technology	1/2
ACM SIGSPATIAL	1/2
Journal of Computer Science and Technology	1/2
Journal of Database Management	1/3
International Symposium on Spatial and Temporal Databases	1/3
International Conference on Mobile Data Management	1/3

在确定语料库训练的具体模型时，基于以下考虑选择了LSTM网络：

(i) 在分类问题中，尽管以CNN表示的前馈网络仍具有性能优势，LSTM 能够真实地表示或模拟人类行为的认知过程、逻辑发展和神经组织，并能够处理复杂任务。

(ii) 语料库中的查询数量并不是特别多，因此不能选择过于复杂的深度学习模型。如果使用参数量较大的复杂模型进行训练，很可能会导致模型欠拟合。与最近提出的 Transformer 和 Bert 模型相比，LSTM 更简单，也更容易实现。

表4 专家评分示例

Natural Language Query Examples	Moving Objects Query	Query Type Support	Entity Integrity	Total Score
Where were the trains at a certain time t1?	0.2	0.4	0.4	1
How many trains at Nanjing station and show me the closest way to the station?	0.2	0.15	0	0.35
Is there any one see the red car?	0	0	0	0
What trains pass the station Nantong during [8am, 9qm]?	0.2	0.2	0.4	1
Show me five nearest neighbors to the train 3 right now.	0.2	0.4	0.25	0.85
When did the train pass underground Nanjing Station?	0.2	0.4	0.3	0.9

5.2 Constructing structured languages

After the semantic understanding, we get the query type and map the entities to the corresponding positions according to defined rules. Finally, we combine the suitable operators to form the final structured sentence. The steps are reported in Fig. 5. The template of the query expression is as follows:

query < data relation > *feed filter* [< **special operators** > < values >] *extend* < **special operators** > [< values >] *consume*;

Extracted entities are mapped to the data relation and corresponding values according to the query type. At the same time, operators are selected to constitute the query structure. Some queries with specific objects need to add *filter* [*<Id> = <number>*] or the id of the object to fill in special operators. We support five types of queries: (i) *time interval queries*, (ii) *range queries*, (iii) *nearest neighbor queries*, (iv) *trajectory similarity queries* and (v) *join queries*. To help explain the procedure, we use an example relation storing underground trains as moving objects. The schema is *Trains*(Id: int, Trip: mpoint). Table 5 summarizes kernel operators for processing moving objects.

The system provides structured queries construction and visual query results for different types of moving objects queries. Each query type requires different entities for natural language query, as shown in Fig. 6.

Definition 1 Time interval queries Given an interval $[t_1, t_2]$, the query looks for objects whose time intervals intersect $[t_1, t_2]$.

In order to compare the query time with a trajectory's time, the operator *deftime* is utilized to extract the defined time interval of a trajectory. The operator *filter* evaluates a query

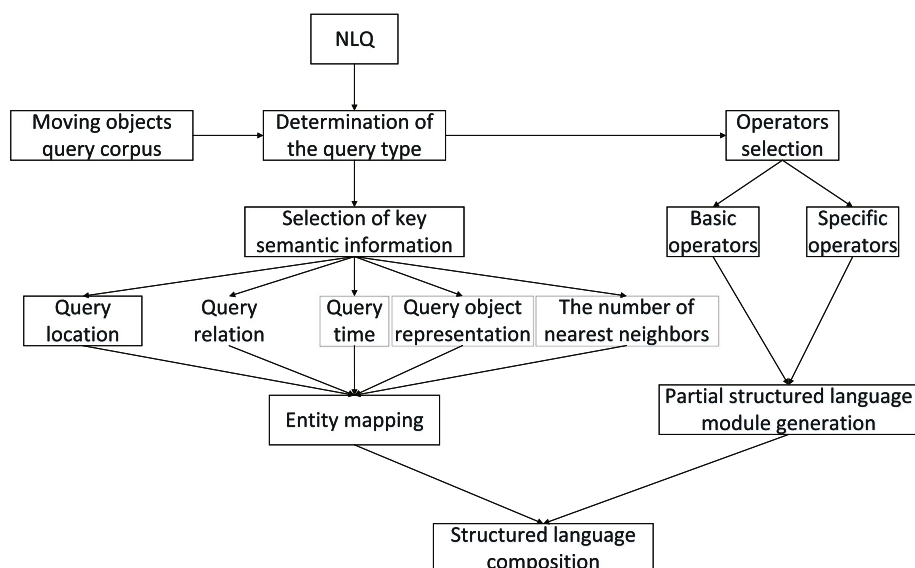


Fig. 5 Building structured language queries

5.2 构造结构化语言

在语义理解之后，我们得到查询类型，并根据定义的规则将实体映射到相应的位置。最后，我们结合合适的运算符形成最终的结构化句子。步骤如图5所示。查询表达式的模板如下：

query < data relation > *feed filter* [< **special operators** > < values >] *extend* < **special operators** > [< values >] *consume*;

提取的实体根据查询类型映射到数据关系和相应的值。同时，选择运算符以构成查询结构。某些具有特定对象的查询需要添加 *filter* [*.<Id> = <number>*] 或对象的 ID 来填充特殊运算符。我们支持五种类型的查询：(i) *time interval queries*, (ii) *range queries*, (iii) *nearest neighbor queries*, (iv) *trajectory similarity queries* 和 (v) *join queries*。为了帮助解释这个过程，我们使用一个存储地下列车作为移动对象的示例关系。该模式为 *Trains*(Id: int, Trip: mpoint)。表 5 总结了处理移动对象的核心运算符。

该系统为不同类型的移动对象查询提供结构化查询构建和可视化查询结果。每种查询类型在自然语言查询中需要不同的实体，如图6所示。

定义 1 时间间隔查询 给定一个间隔 $[t_1, t_2]$ ，查询寻找其时间间隔与 $[t_1, t_2]$ 相交的对象。

为了将查询时间与轨迹的时间进行比较，使用操作符 *deftime* 来提取轨迹的定义时间间隔。操作符 *filter* 用于评估查询

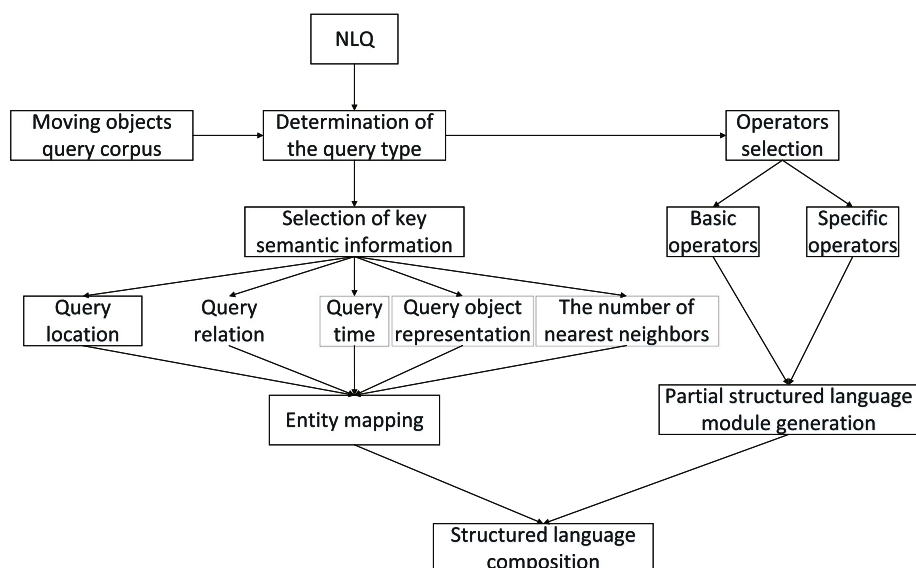


图5 构建结构化语言查询

Table 5 Operators for moving objects

Operator	Signature
trajectory	$mpoint \rightarrow line$
atperiods	$mpoint \times periods \rightarrow mpoint$
present	$mpoint \times periods (instant) \rightarrow bool$
deftime	$mpoint \rightarrow periods$
at	$mpoint \times region \rightarrow mpoint$
passes	$mpoint \times region (point) \rightarrow bool$
atinstant	$mpoint \times instant \rightarrow ipoint$
inst	$ipoint \rightarrow instant$
val	$ipoint \rightarrow point$
intersects	$xrectangle \times xrectangle \rightarrow bool$

predicate and returns objects fulfilling the condition. For this query, an intersecting between two time intervals is evaluated. The template of the query expression is as follows in which the content in $\langle \rangle$ will be extracted from natural language expressions.

query $\langle data\ relation \rangle\ feed\ filter[deftime(Trip)intersects\ \langle query\ time \rangle]$
consume;

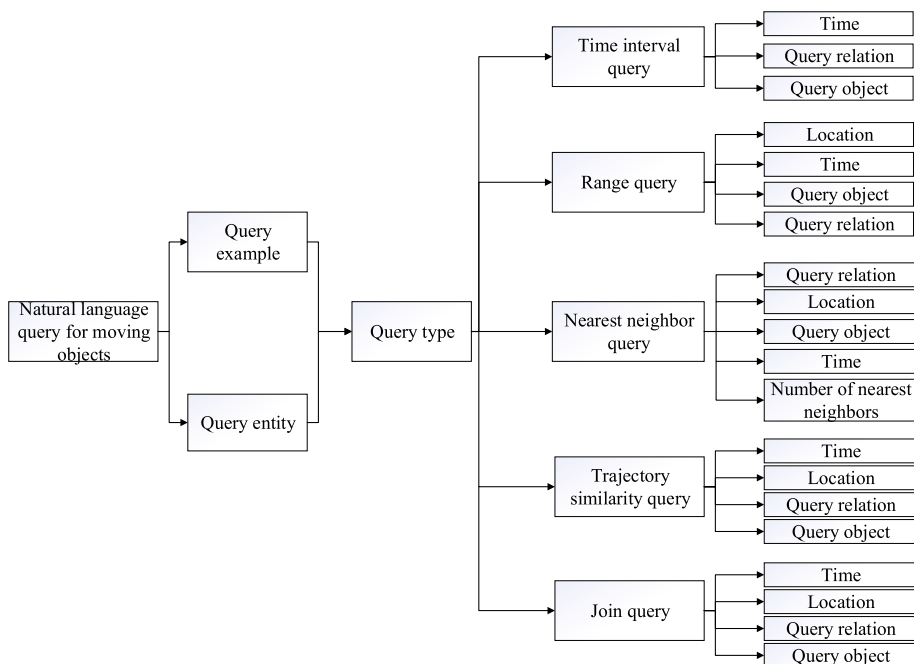


Fig. 6 Query required entities

表5 移动物体的操作符

Operator	Signature
trajectory	$mpoint \rightarrow line$
atperiods	$mpoint \times periods \rightarrow mpoint$
present	$mpoint \times periods (instant) \rightarrow bool$
deftime	$mpoint \rightarrow periods$
at	$mpoint \times region \rightarrow mpoint$
passes	$mpoint \times region (point) \rightarrow bool$
atinstant	$mpoint \times instant \rightarrow ipoint$
inst	$ipoint \rightarrow instant$
val	$ipoint \rightarrow point$
intersects	$xrectangle \times xrectangle \rightarrow bool$

谓词并返回满足条件的对象。对于此查询，将评估两个时间区间的交集。查询表达式的模板如下，其中 <> 中的内容将从自然语言表达式中提取。

```
query < data relation > feed filter[deftime(Trip)intersects < query time >]
consume;
```

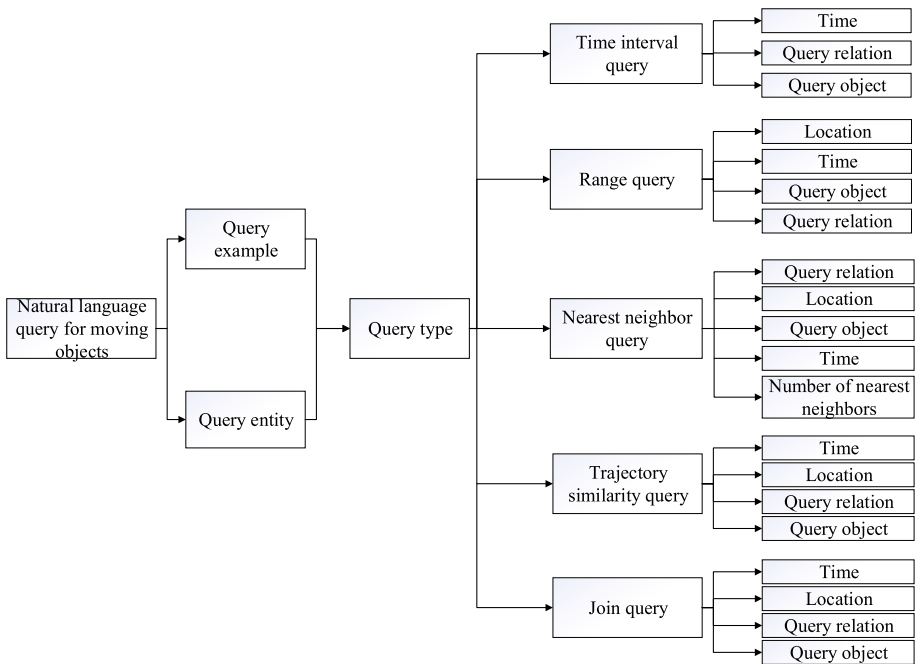


图6 查询所需实体

Assuming the query time is “[7:00, 8:00] on November 20, 2020”, we get the following structured language:

```
query Trains feed filter[deftime(.Trip) intersects[const periods value
("2020 - 11 - 20 - 7 : 00" "2020 - 11 - 20 - 8 : 00" TRUE TRUE)]] consume;
```

There is another approach to accomplish the task by evaluating whether an object is defined at the query time. The operator *atperiod* restricts the movement to a time interval and the operator *present* determines whether an object is valid in a certain time interval. The template is:

```
query < data relation > feed filter[present(.Trip intersects < query
time >)] consume;
```

Definition 2 Spatio-temporal range queries Given a spatio-temporal window Q and a set of trajectories O , the query returns a set of trajectory O' , and for $\forall O' \subset O$, O' intersects $Q(t, r)$.

The special operator *intersect* finds objects that satisfy both time and space conditions. The operator *intersect* can be used to check whether two rectangles cross each other. The operator *deftime* can get the time interval when the moving object is limited in the specific space condition. We use the operation *intersects* to get the intersection between the limited space and the limited time to construct the query. Assuming that the location is “Tiergarten” and the time interval is November 20, 2020 from 8:00 to 10:00, we query trains that meet two conditions above:

```
query Trains feed filter[(deftime(.Trip at Tiergarten)intersects
[const periods value(("2020 - 11 - 20 - 8 : 00" "2020 - 11 - 20 - 10 : 00"
TRUE TRUE))]]]consume;
```

When the query does not specify the specific query object identifier, the structured expression is as follows:

```
query < relation > feed filter [(deftime(. < Trip > at < region >)
intersects < period >)]consume;
```

Definition 3 Nearest Neighbor queries Given a query mq , a distance set D and a relation E with an attribute MO of type $mpoint$, the query which holds all the distances between the queried moving objects returns a subset $E' \subset E$, where each tuple meets the condition that for the tuple $e \in E'$, there exists an instance of time t . The distance $d(e.MO(t), mq(t)) \in D$ is among the k smallest distances.

We use the operator *knearest* for nearest neighbor query. Before querying, we need to create a unit representation of the moving object. The relation $Trains(Id:int, Trip:mpoint)$ is used as an example:

```
let UnitsTrains = Trains feed projectextendstream[Id, Line, Up; UTrip :
units(.Trip)]consume;
```

The operator *projectextendstream* creates a unit stream from the *Trip* attribute of each input tuple, and outputs a copy of the original tuple for each unit, which is limited to attributes *Id*, *Line*, *Up* and the new unit attribute *UTrip*. Parameters of the operator *knearest* are ordered

假设查询时间是 “[7:00, 8:00] on November 20, 2020” , 我们得到以下结构化语言 :

```
query Trains feed filter[deftime(.Trip) intersects[const periods value
("2020 - 11 - 20 - 7 : 00" "2020 - 11 - 20 - 8 : 00" TRUE TRUE)]] consume;
```

还有另一种方法可以完成任务, 即通过评估一个对象在查询时间是否已定义。操作符 *atperiod* 将移动限制在一个时间间隔内, 操作符 *present* 判断一个对象在某个时间间隔内是否有效。模板是:

```
query < data relation > feed filter[present(.Trip intersects < query
time >)] consume;
```

定义 2 时空范围查询 给定一个时空窗口 Q 和一组轨迹 O , 查询返回一组轨迹 O' , 并且对于 $\forall O' \subset O$, O' 与 $Q(t, r)$ 相交。

特殊运算符 *intersect* 用于查找同时满足时间和空间条件的物体。运算符 *intersect* 可用于检查两个矩形是否相互交叉。运算符 *deftime* 可以获取移动物体在特定空间条件下的时间区间。我们使用操作 *intersects* 来获取有限空间与有限时间之间的交集以构建查询。假设位置为 “Tiergarten”, 时间区间为 2020 年 11 月 20 日 8:00 至 10:00, 我们查询满足上述两个条件的列车:

```
query Trains feed filter[(deftime(.Trip at Tiergarten)intersects
[const periods value(("2020 - 11 - 20 - 8 : 00" "2020 - 11 - 20 - 10 : 00"
TRUE TRUE)])])consume;
```

当查询未指定具体的查询对象标识符时, 结构化表达如下:

```
query < relation > feed filter [(deftime(. < Trip > at < region >)
intersects < period >)]consume;
```

定义 3 最近邻查询 给定一个查询 mq 、一个距离集合 D 和一个包含 *mpoint* 类型属性 MO 的关系 E , 查询返回所有被查询移动对象之间的距离的子集 $E' \subset E$, 其中每个元组满足条件: 对于元组 $e \in E'$, 存在一个时间实例 t , 距离 $d(e.MO(t), mq(t)) \in D$ 属于 k 个最小距离之内。

我们使用操作符 *knearest* 进行最近邻查询。在查询之前, 我们需要创建移动对象的单元表示。关系 *Trains*(*Id*:int, *Trip*:*mpoint*) 用作示例:

```
let UnitsTrains = Trains feed projectextendstream[Id, Line, Up; UTrip :
units(.Trip)]consume;
```

操作符 *projectextendstream* 从每个输入元组的 *Trip* 属性创建一个单元流, 并为每个单元输出原始元组的副本, 其限制为属性 *Id*、*Line*、*Up* 和新的单元属性 *UTrip*。操作符 *knearest* 的参数是有序的

stream of unit tuples, the name of the attribute a , the queried moving object and the number of neighbors k .

The operator *knearest* returns a stream of tuples with the same structure as the input stream. However, the output units in attribute belong to the k nearest moving points represented in the input stream and are limited by the time when objects arrive between the k nearest moving points, and requires the first parameter stream to arrive in the order of units' start time. We create a version of *UnitTrains*, where units are sorted by start time:

*let UT Ordered = UnitTrains feed extend[Mintime :
minimum(deftime(.UTrip))] sortby[Mintime asc] consume;*

We can get the final structured language by using *UTOrdered*:

*query UTOrdered feed filter [(deftime(.UTrip) intersects < period >)]
knearest[UTrip, < object >, < k >]consume;*

Definition 4 Trajectory similarity queries Given a set of trajectories denoted by O , the query returns the trajectory $o_i \in O$, of which the similarity distance measure m_i is the smallest during the same point t or time interval $[t_1, t_2]$.

We use the operator *atperiod* to limit the movement to a certain period of time, or use the operator *atinstant* to calculate the moving point at a given moment, and return the moving point in a certain period of time or at a certain moment. Then we use the operator *deftime* to project the motion to the time dimension. If we need to limit the number of objects, we can use the operator *head*. Suppose the time interval is from 9:00 to 10:00 on November 20, 2020, we query the subway pairs with cross trajectories, and take the first ten objects for comparison. We use the operator *symmjoin* and the operator *filter* to find out the moving objects that have appeared in a certain period of time and have trajectories crossing, and then project the new line value attributes together with the original partial attributes:

*query UTOrdered feed filter [(deftime(.UTrip) intersects < period >)]
knearest[UTrip, < object >, 1]consume;*

Definition 5 Join queries Given a set of trajectories denoted by O , the query returns the trajectory $o_i \in O$, which is shared by the queried moving objects during time interval $[t_1, t_2]$.

If we can find the closest moving object to the query object within a period of time, we can judge that the trajectories of the two moving objects are similar. The operator *knearest* looks for the nearest neighbors in a trajectory segment, but multiple trajectory segments may have appeared in the time interval of a query. We count each nearest neighbor, and the object that has the most frequent occurrence in the time interval is judged to be similar. If the number of occurrences is the same, the longest trajectory has priority. Using *UTOrdered* sorted by start time, we get the final structured language:

*query Trains feed filter[.Up] extend[Stretch : trajectory(.Trip atperiods
[const periods value ("2003 - 11 - 20 - 9 : 00" "2003 - 11 - 20 - 09 : 10"
TRUE TRUE))]] extend[T : deftime(.Trip)] head[10] t1 Trains feed
filter[.Up]extend[Stretch : trajectory(.Trip atperiods [const periods value
("2020 - 11 - 20 - 9 : 00" "2020 - 11 - 20 - 09 : 10" TRUE TRUE))]]]*

单位元组的流, 属性名 a , 查询的移动对象以及邻居数量 k 。

运算符 *knearest* 返回一个具有与输入流相同结构的元组流。然而, 输出单元在属性中属于输入流中表示的 k 最近移动点, 并且受限于对象在 k 最近移动点之间到达的时间, 并且要求第一个参数流按单元的开始时间顺序到达。我们创建了一个 *UnitTrains* 版本, 其中单元按开始时间排序:

```
let UTOrdered = UnitTrains feed extend[Mintime :
minimum(deftime(.UTrip))] orderby[Mintime asc] consume;
```

我们可以通过使用 *UTOrdered* 得到最终的结构化语言:

```
query UTOrdered feed filter [(deftime(.UTrip) intersects < period >)]
knearest[UTrip, < object >, < k >]consume;
```

定义 4 轨迹相似性查询 给定一组由 O 表示的轨迹, 查询返回轨迹 $o_i \in O$, 其在相同点 t 或时间区间 $[t_1, t_2]$ 内的相似性距离度量 m_i 最小。

我们使用运算符 *atperiod* 将移动限制在特定时间段内, 或者使用运算符 *atinstant* 计算某一时刻的移动点, 并返回某一时间段或某一时刻的移动点。然后我们使用运算符 *deftime* 将运动投影到时间维度。如果我们需要限制对象数量, 可以使用运算符 *head*。假设时间间隔是 2020 年 11 月 20 日 9:00 到 10:00, 我们查询轨迹交叉的地铁对, 并取前十个对象进行比较。我们使用运算符 *symmjoin* 和运算符 *filter* 找出在特定时间段内出现且轨迹交叉的移动对象, 然后将新的线路数值属性与原有部分属性一起投影:

```
query UTOrdered feed filter [(deftime(.UTrip) intersects < period >)]
knearest[UTrip, < object >, 1]consume;
```

定义 5 连接查询 给定一组用 O 表示的轨迹, 该查询返回轨迹 $o_i \in O$, 该轨迹是所查询的移动对象在时间区间 $[t_1, t_2]$ 内共有的。

如果我们能在一段时间内找到查询对象最近的移动对象, 我们就可以判断两个移动对象的轨迹是相似的。操作符 *knearest* 用于在轨迹片段中寻找最近邻, 但在查询的时间间隔内可能出现多个轨迹片段。我们统计每个最近邻, 时间间隔内出现次数最多的对象被判定为相似。如果出现次数相同, 则以轨迹最长者为优先。使用按开始时间排序的 *UTOrdered*, 我们得到最终的结构化语言:

```
query Trains feed filter[.Up] extend[Stretch : trajectory(.Trip atperiods
[const periods value ("2003 - 11 - 20 - 9 : 00" "2003 - 11 - 20 - 09 : 10"
TRUE TRUE))]] extend[T : deftime(.Trip)] 头[10] t1 Trains feed
filter[.Up]extend[Stretch : trajectory(.Trip atperiods [const periods value
("2020 - 11 - 20 - 9 : 00" "2020 - 11 - 20 - 09 : 10" TRUE TRUE))]]]
```

```

extend[T : deftime(.Trip)] head[10] t2 symmjoin
[.Stretch_t1 intersects1..Stretch_t2] filter[.T_t1 intersects .T_t2] consume;

```

In the expression, the operator relation represents the relation of the query, the operator region represents the limited region, and the operator period represents the limited time period. This type of query does not specify the identity of the specific query object. The expression is as follows:

```

query <relation> feed filter[.<Trip> present <period>] extend[Stretch :
trajectory(. <Trip> atperiods <period>)] head[n] t1 <relation> feed
filter[.<Trip> present <period>] extend[Stretch : trajectory(. <Trip>
atperiods <period>)] head[n] t2 symmjoin [<period> intersects1
<period>] filter[.<period> intersects <period>] consume;

```

6 Experimental evaluation

The evaluation is conducted in a desktop PC (Intel(R) Core(TM) i5-10210U CPU, 1.60 GHz, 8 GB memory, 512 GB disk) running Ubuntu 20.04 (64 bits, GNOME version 3.36.8). We develop the proposal in C/C++ and integrate the implementation into an extensible database system SECONDO. Our system has been developed in a prototype system for translating natural language queries into structured query languages [50].

6.1 Datasets

To the best of our knowledge, there is no benchmark of moving objects queries in natural languages. Therefore, we generate two datasets named Moving Objects Queries (MOQ) and Berlin Trains (BT), respectively.

MOQ contains 280 moving objects queries divided into five categories: (i) time interval queries, (ii) range queries, (iii) nearest neighbor queries, (iv) trajectory similarity queries and (v) join queries. These queries are extracted from more than 60 conference and journal papers in the domain of moving objects and then manually processed by experts. Specifically, we select high-quality queries from the corpus to construct MOQ. MOQ is used to test whether different kinds of moving object queries can get correct structured languages. BT contains moving underground trains and is used to evaluate whether the converted structured sentences can provide expected results for users. Some queries can be seen in the [Appendix](#), and these queries satisfy the diversity of expressions as much as possible.

6.2 Measurement

NALMO is designed for users who are not experts in writing query expressions for moving objects. Three criteria are defined to evaluate the translation quality: (i) *effectiveness*, (ii) *time cost* and (iii) *usability*. Effectiveness mainly contains two measurements: *accuracy* and *precision*.

Effectiveness NALMO is designed for users to conveniently query and obtain query results. Whether executable languages can be generated and whether users can get expected results are important factors. The way to verify the effectiveness is to check whether natural language

extend[*T* : *deftime*(*Trip*)] 头[10] *t2* 对称连接

[*Stretch_t1* 与 *1..Stretch_t2*] *filter*[*T_t1 intersects .T_t2*] *consume*;

在表达式中, 操作符 *relation* 表示查询的关系, 操作符 *region* 表示有限的区域, 操作符 *period* 表示有限的时间段。这种类型的查询不指定具体查询对象的身份。表达式如下:

```
query <关系> feed filter[.<Trip> present <时期>] extend[Stretch :
trajectory(. <Trip> 在时期 <时期>)] 头[n] t1 <relation> feed
filter[.<Trip> present <period>] extend[Stretch : trajectory(. <Trip>
atperiods <period>)] 头[n] t2 对称连接 [<period> intersects1
<period>] filter[<period> 相交 <period>] consume;
```

6 实验评估

评估是在一台桌面电脑 (Intel(R) Core(TM) i5-10210U CPU, 1.60 GHz, 8 GB 内存, 512 GB 磁盘) 上进行的, 该电脑运行 Ubuntu 20.04 (64 位, GNOME 版本 3.36.8)。我们使用 C/C++ 开发该方案, 并将实现集成到可扩展的数据库系统 SECONDO 中。我们的系统已在一个用于将自然语言查询翻译为结构化查询语言的原型系统中开发完成 [50]。

6.1 数据集

据我们所知, 目前还没有关于自然语言中移动物体查询的基准。因此, 我们分别生成了两个数据集, 命名为移动物体查询 (MOQ) 和柏林火车 (BT)。

MOQ 包含 280 个移动对象查询, 分为五类: (i) 时间间隔查询, (ii) 范围查询, (iii) 最近邻查询, (iv) 轨迹相似度查询和 (v) 连接查询。这些查询提取自 60 多篇关于移动对象领域的会议论文和期刊论文, 然后由专家手工处理。具体而言, 我们从语料库中选择高质量查询来构建 MOQ。MOQ 用于测试不同类型的移动对象查询是否能够获得正确的结构化语言。BT 包含移动的地铁列车, 用于评估转换后的结构化句子是否能够为用户提供预期的结果。一些查询可以在附录中看到, 这些查询尽可能满足表达的多样性。

6.2 测量

NALMO 是为不擅长为移动对象编写查询表达式的用户设计的。定义了三个标准来评估翻译质量: (i) *effectiveness*, (ii) *time cost* 和 (iii) *usability*。有效性主要包含两个测量指标: *accuracy* 和 *precision*。

效果 NALMO 旨在让用户方便地查询并获取查询结果。是否能够生成可执行语言以及用户是否能够得到预期结果是重要因素。验证其有效性的方法是检查自然语言是否

expressions are correctly and precisely translated into structured query languages, that is, *accuracy*. The expression of accuracy is shown below.

$$accuracy = \frac{\#executable}{\#overall \text{ queries}} \quad (4)$$

Such a value is defined as the ratio of the number of correctly translated sentences to the overall number of sentences. MOQ is used for test. In the evaluation, domain experts' judgement is considered to determine whether the produced structured query correctly corresponds to the natural language query. However, only using the accuracy is not sufficient to verify the effectiveness of NLIDB systems. This is because users may be not concerned about the generation of structured sentences but pay attention to whether the outputs are the expected results. Motivated by this, we use the *precision* to further evaluate the translation effect. The expression of precision is shown below.

$$precision = \frac{\#expected \text{ results}}{\#overall \text{ queries}} \quad (5)$$

The *precision* is defined as the ratio of the number of queries producing the expected results in the system to the overall number of queries. BT is used for test. In some cases, the natural language expression is unclear or the key information is missing. Therefore, NALMO generates a structured query, but the query fails to output the expected results.

Time cost We do the evaluation by considering (i) *the overall time cost* and (ii) *the time cost in each module*, including parsing, analyzing and constructing the structured languages. We take the average value over three runnings as the final result.

Usability NALMO is designed for ordinary users who do not know structured sentences. User experience is a very important indicator. We recruite four volunteers and conduct three rounds of independent tests to investigate user satisfaction.

Furthermore, when training the model that recognizes the query type, we define a LSTM sequence model. The dimension of the Embedding layer of the model is set to 64, and the number of memory cells of the LSTM layer is set to 128. The output layer of the model is a fully connected layer with five categories.

6.3 Experimental results

6.3.1 Accuracy and precision

We need to conduct a semantic understanding test for each query to determine whether the system can provide the entity information for the query. First, the expert gives a comparison table of the entities contained in each moving objects query, and scores each processed query according to this comparison table. Each query has a full score of 1, and for each missing entity, a certain score will be deducted proportionally. For example, a spatio-temporal range query contains four types of information: time information, location information, query relation and query object. For each missing category of information, a quarter of the point is deducted, which is 0.25 points, and so on. The test is performed according to five query categories, including time period query, range query, nearest neighbor query, trajectory similarity query and join query. We use the average score of each query types as the final results, as shown in Fig. 7.

表达式被正确且精确地翻译成结构化查询语言，即 *accuracy*。准确性的表达如下所示。

$$accuracy = \frac{\#executable}{\#overall \ queries} \quad (4)$$

这样的值被定义为正确翻译的句子数与句子总数的比值。MOQ 用于测试。在评估中，会考虑领域专家的判断来确定生成的结构化查询是否正确对应自然语言查询。然而，仅仅使用准确率不足以验证 NLIDB 系统的有效性。这是因为用户可能不关心结构化句子的生成，而关注输出是否为预期结果。有鉴于此，我们使用 *precision* 来进一步评估翻译效果。精确度的表达如下所示。

$$precision = \frac{\#expected \ results}{\#overall \ queries} \quad (5)$$

precision 被定义为系统中产生预期结果的查询数量与查询总数的比率。BT 用于测试。在某些情况下，自然语言表达不清或关键信息缺失。因此，NALMO 生成了结构化查询，但查询未能输出预期结果。

时间成本 我们通过考虑 (i) *the overall time cost* 和 (ii) *the time cost in each module* 来进行评估，包括解析、分析和构建结构化语言。我们取三次运行的平均值作为最终结果。

可用性 NALMO 是为不懂结构化句子的普通用户设计的。用户体验是一个非常重要的指标。我们招募了四名志愿者，并进行了三轮独立测试以调查用户满意度。

此外，在训练识别查询类型的模型时，我们定义了一个LSTM序列模型。模型的嵌入层维度设置为64，LSTM层的记忆单元数量设置为128。模型的输出层是一个具有五个类别的全连接层。

6.3 实验结果

6.3.1 准确性与精密度

我们需要对每个查询进行语义理解测试，以确定系统是否能够提供该查询的实体信息。首先，专家给出每个移动对象查询中包含的实体的对比表，并根据该对比表对每个处理过的查询进行评分。每个查询的满分为1分，对于每缺少一个实体，将按比例扣除相应的分数。例如，一个时空范围查询包含四类信息：时间信息、位置信息、查询关系和查询对象。每缺少一类信息，扣除四分之一的分数，即0.25分，依此类推。测试按照五类查询进行，包括时间段查询、范围查询、最近邻查询、轨迹相似性查询和关联查询。我们使用每类查询的平均分作为最终结果，如图7所示。

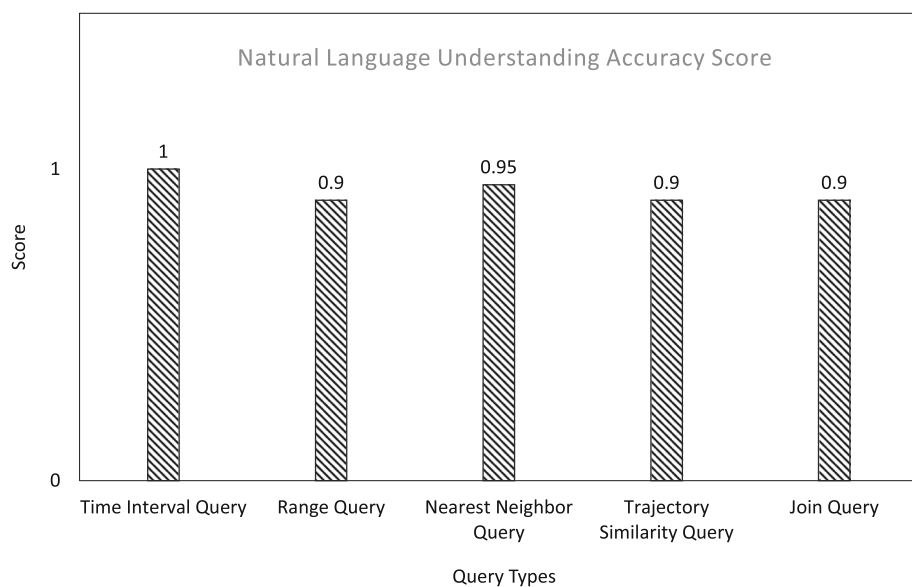


Fig. 7 Natural language understanding accuracy score

In general, different types of scores are basically the same, but in detail, the score of each query category is still different. The time interval query gets the highest score, because the time interval query only needs to concentrate on time information without any other information. We can easily judge and construct time interval query. Other types of queries not only consider time information, but also need to consider other entity information such as location and the number of nearest neighbors.

Figures 8 and 9 show the effectiveness. The overall accuracy of NALMO is 96.8%. Among 9 wrong queries, there are 2 time point queries, 2 time interval queries, 1 spatio-temporal range query, 2 trajectory similarity queries and 2 join queries. The time format of 4 expressions cannot be recognized by spaCy, and NALMO terminates the constructing procedure. There are 3 queries vaguely expressing the time information. Specifically, the word “*right now*”

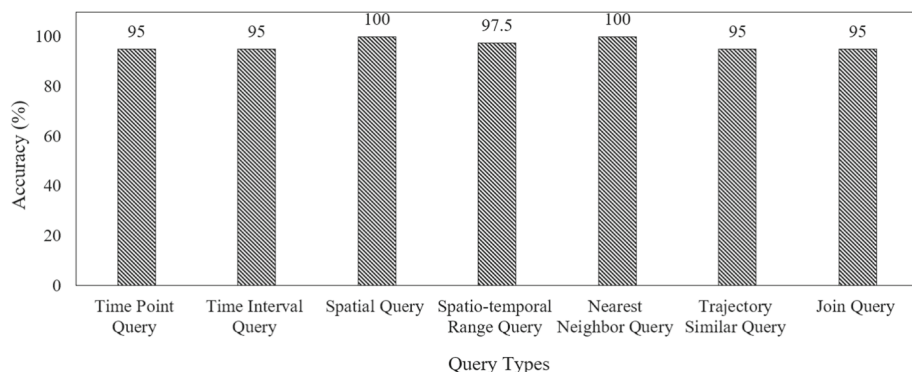


Fig. 8 Accuracy

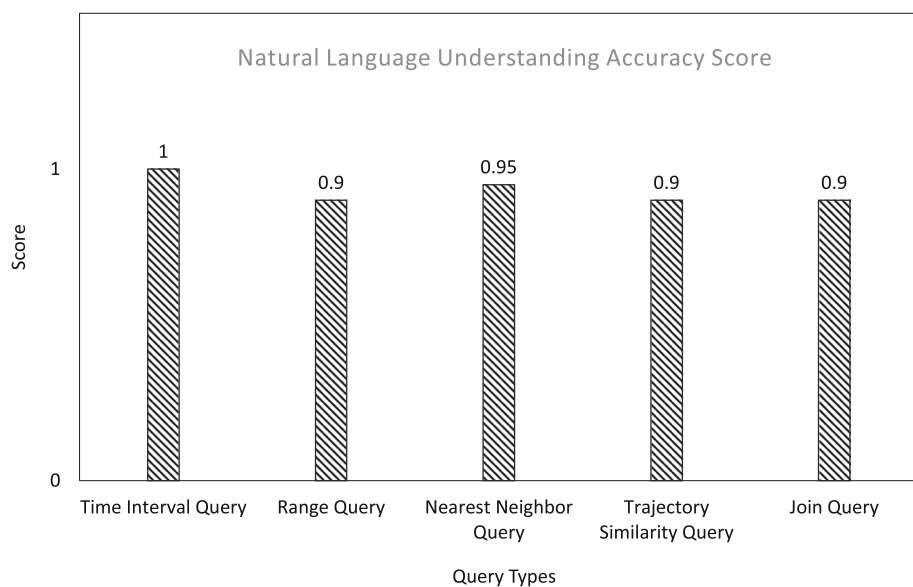


图 7 自然语言理解准确率

一般来说，不同类型的分数基本相同，但在细节上，每个查询类别的分数仍然不同。时间间隔查询得分最高，因为时间间隔查询只需关注时间信息而不考虑其他信息。我们可以轻松判断和构建时间间隔查询。其他类型的查询不仅需要考虑时间信息，还需要考虑其他实体信息，如位置和最近邻的数量。

图8和图9展示了其有效性。NALMO的整体准确率为96.8%。在9个错误查询中，有2个时间点查询、2个时间区间查询、1个时空范围查询、2个轨迹相似性查询和2个连接查询。4个表达式的时间格式无法被spaCy识别，NALMO因此终止了构建过程。有3个查询模糊地表达了时间信息。具体来说，单词 “*right now*”

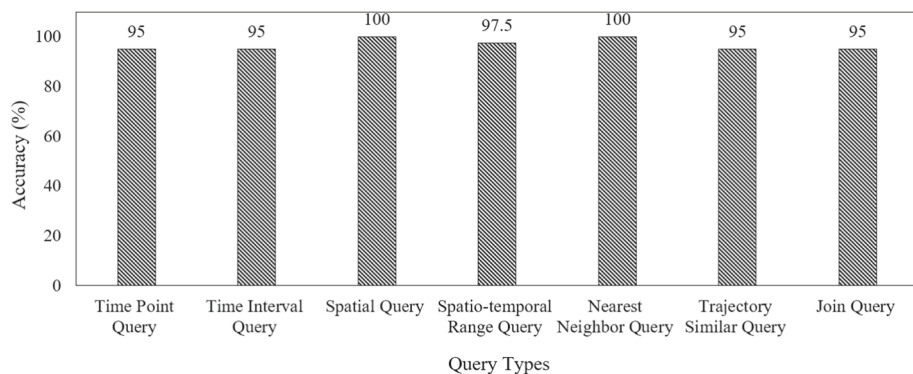


图 8 精度

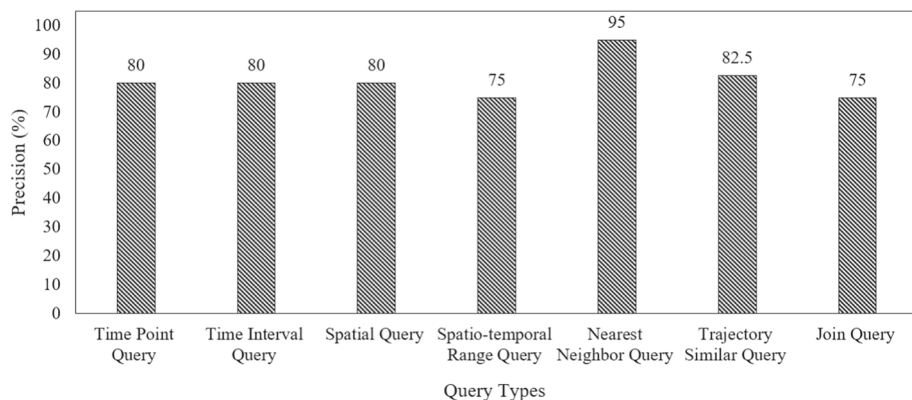


Fig. 9 Precision

in “Where are the trains right now?” cannot be recognized by NALMO as moving objects queries require accurate time information. In trajectory similarity queries, we only support returning the most similar trajectory at the moment but not k (> 2) similar trajectories.

The overall precision is 81.1%. The value is lower than accuracy. This is because some queries are translated into executable structured languages but did not report expected results. Among 53 wrong queries, the time point query contains 8 wrong sentences. The reason is that the relation in the structured sentence is *Planes* which does not exist in the dataset. The time interval query contains 8 wrong sentences among which 4 are spatio-temporal range queries and 2 queries do not find required relations. The spatial query contains 8 wrong sentences. There are 6 queries without locations in the knowledge base such that structured languages are not executable in the database. There are 2 queries looking for non-existent objects. For example, “train G55” in “Where was the train G55 at 12:00 o’clock?” is an object that cannot be queried. The spatio-temporal range query contains 10 wrong sentences among which 6 queries have locations not in the knowledge base and 4 queries do not have required relations. The trajectory similarity query contains 7 wrong sentences. The queries are answered by using algorithms of nearest neighbor queries. The nearest neighbor query contains 2 wrong sentences both of which are identified as similarity queries. The join query contains 10 wrong sentences among which 4 are nearest neighbor queries or trajectory similarity queries, and 4 do not contain suitable locations. The dataset size has a significant impact on the precision of users’ expected query results.

6.3.2 Time cost

In addition, a location-based prefix tree is constructed to improve the extraction efficiency of location information. The location matching of the system is one-to-one matching before. Using the index, the overall efficiency has been greatly improved, especially when the amount of data is relatively large, the efficiency improvement is more obvious. The specific results are shown in Fig. 10.

The time costs are reported in Fig. 11. Nearest neighbor queries, trajectory similarity queries and join queries take more time than other queries, the average time of which are 1.64s, 1.68s and 1.67s, respectively. The three queries consider the time, location, query object identifier, and the number of nearest neighbors, requiring more entities than other queries. The time cost includes two parts: (i) *semantic parsing and analysis*, and (ii) *the*

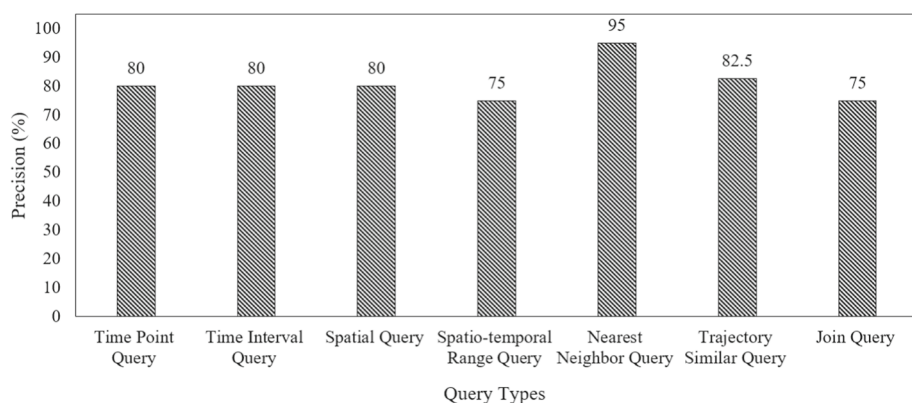


图9 精度

在“Where are the trains right now?”中无法被 NALMO 识别为移动对象，查询需要准确的时间信息。在轨迹相似性查询中，我们目前只支持返回最相似的轨迹，而不支持 $k (> 2)$ 条相似轨迹。

整体精确率为81.1%。该数值低于准确率，这是因为一些查询被翻译成可执行的结构化语言，但未返回预期结果。在53个错误查询中，时间点查询包含8个错误句子。原因是结构化句子中的关系是Planes，该关系在数据集中不存在。时间区间查询包含8个错误句子，其中4个是时空范围查询，2个查询未找到所需关系。空间查询包含8个错误句子。有6个查询在知识库中没有地点信息，因此结构化语言在数据库中不可执行。有2个查询寻找不存在的对象。例如，“train G55”在

“Where was the train G55 at 12:00 o'clock?”中是一个无法查询的对象。时空范围查询包含10个错误句子，其中6个查询的地点不在知识库中，4个查询没有所需关系。轨迹相似查询 cont

6.3.2 时间成本

此外，构建了一个基于位置的前缀树，以提高位置信息的提取效率。系统之前的位置匹配是单对单匹配。使用索引后，总体效率得到了大幅提升，尤其是在数据量相对较大时，效率提升更加明显。具体结果如图10所示。

时间成本如图11所示。最近邻查询、轨迹相似性查询和连接查询比其他查询花费更多时间，其平均时间分别为1.64秒、1.68秒和1.67秒。这三种查询考虑了时间、位置、查询对象标识符以及最近邻数量，因此需要的实体比其他查询更多。时间成本包括两个部分：(i) *semantic parsing and analysis*，以及 (ii) *the*

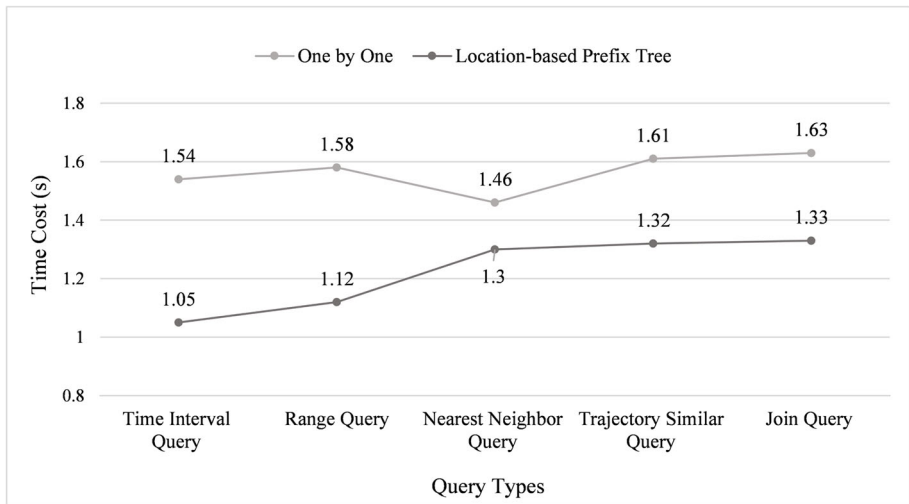


Fig. 10 Natural language understanding time cost

determination of query types and the combination of structured languages. For the seven types of queries in Fig. 11, the average time cost of the entire translation process is about 1.49s, the average time cost of the natural language understanding part is about 1.18s, and the average time cost of the query translation part is about 0.31s. The main part of time is spent on semantic parsing and analysis. The parsing time is affected by the speed of natural language processing tools to a certain extent.

The performance of the overall translation process is evaluated by scaling the size of the dataset. We first extend the *berlintest* database in *SECONDO* by increasing the number of its moving object relations. Then, queries in *MOQ* are divided into five groups: (i) 10 queries

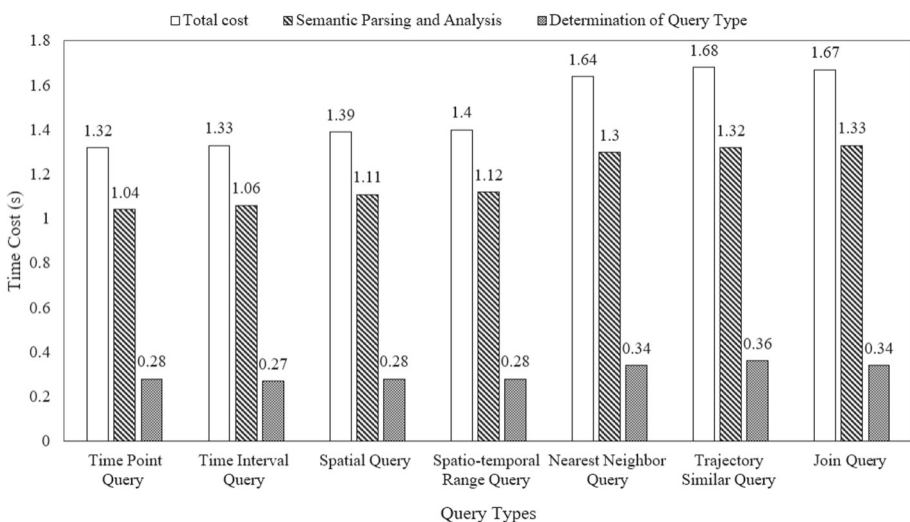


Fig. 11 Time cost

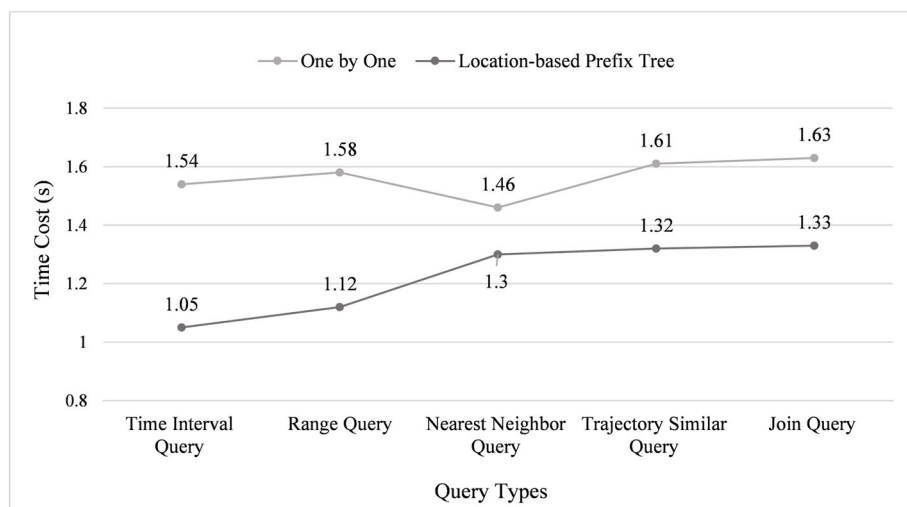


图10 自然语言理解时间成本

determination of query types and the combination of structured languages对于图 11 中的七种查询类型，整个翻译过程的平均耗时约为 1.49 秒，自然语言理解部分的平均耗时约为 1.18 秒，查询翻译部分的平均耗时约为 0.31 秒。耗时的主要部分用于语义解析和分析。解析时间在一定程度上受自然语言处理工具速度的影响。

通过缩放数据集的规模来评估整体翻译过程的性能。我们首先通过增加其移动对象关系的数量来扩展 SECONDO 中的 *berlintest* 数据库。然后，将 MOQ 中的查询分为五组：(i) 10 个查询

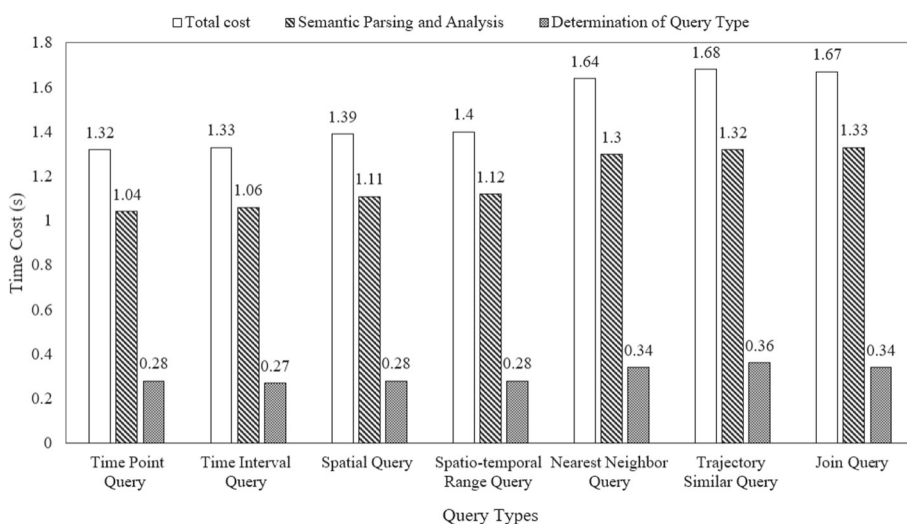


图11 时间成本

for Group A, (ii) 20 queries for Group B, (iii) 50 queries for Group C, (iv) 100 queries for Group D, (v) 280 queries for Group E. The total time cost of the translation process of each group is measured separately. The experimental results are shown in Fig. 12. The time cost of each group is in the range of 1.49 ± 0.01 s. We conclude that the time cost of the translation process is independent of the size of the test dataset.

6.3.3 Accuracy, precision and time cost between template matching and model

When we use template matching instead of the trained model to determine the query type, accuracy and precision decrease. Time cost is almost the same as before. Table 6 shows that the overall accuracy and precision are 90% and 84.8%, respectively, which are significantly lower than using the trained model. In terms of accuracy, there is no change in time point query and time period query. Spatial query and spatio-temporal range query all add 6 new wrong queries. Although the location is not included in the location knowledge base, the model can still determine the query type and structured queries can be generated. However, template matching lacks location information to directly generate structured queries. The nearest neighbor query, trajectory similarity query and join query add 8, 7 and 4 wrong sentences, respectively. This is because nearest neighbor query, trajectory similarity query and join query are matched by keywords such as “*neighbor*”, “*similar*” and “*same*”. Due to the limitations of template matching, when the keywords do not appear in the sentence, the query cannot be recognized. In terms of precision, there is no change in time point query and time interval query. Spatial query and spatio-temporal range query remain unchanged. This is because the use of model determination does not generate structured sentences, and NALMO will not give user expected results. The wrong queries of nearest neighbor query, trajectory similarity query and join query all do not generate structured sentences. In terms of time cost, there is not much difference compared with using the model which has little effect on user experience.

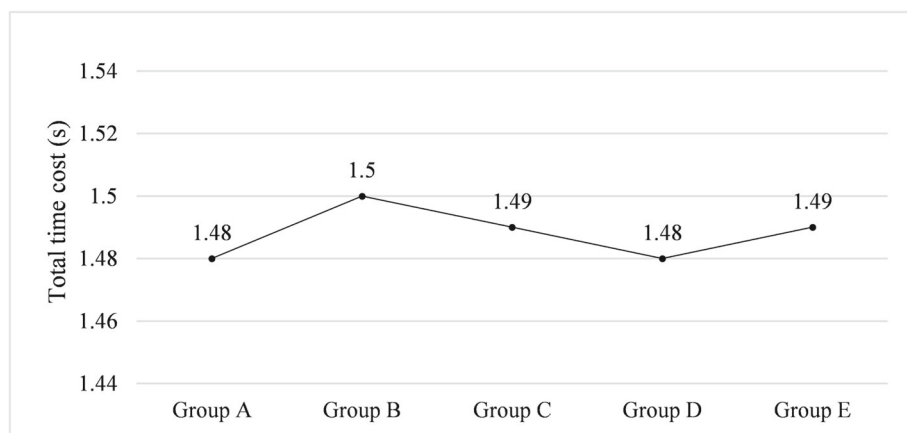


Fig. 12 The total time cost of the entire translation process

对于A组, (ii) B组20个查询, (iii) C组50个查询, (iv) D组100个查询, (v) E组280个查询。每组翻译过程的总时间成本分别进行测量。实验结果如图12所示。每组的时间成本在 1.49 ± 0.01 秒的范围内。我们得出结论, 翻译过程的时间成本与测试数据集的大小无关。

6.3.3 模板匹配与模型之间的准确性、精度和时间成本

当我们使用模板匹配而不是训练好的模型来确定查询类型时, 准确性和精确度会下降。时间成本几乎与之前相同。表6显示, 总体准确率和精确率分别为90%和84.8%, 显著低于使用训练模型的情况。在准确性方面, 时间点查询和时间段查询没有变化。空间查询和时空范围查询各增加了6条错误查询。尽管位置不包含在位置知识库中, 模型仍然可以确定查询类型并生成结构化查询。然而, 模板匹配缺乏位置信息, 无法直接生成结构化查询。最近邻查询、轨迹相似查询和连接查询分别增加了8条、7条和4条错误句子。这是因为最近邻查询、轨迹相似查询和连接查询是通过 “*neighbor*”、“*similar*” 和 “*same*” 等关键词来匹配的。由于...

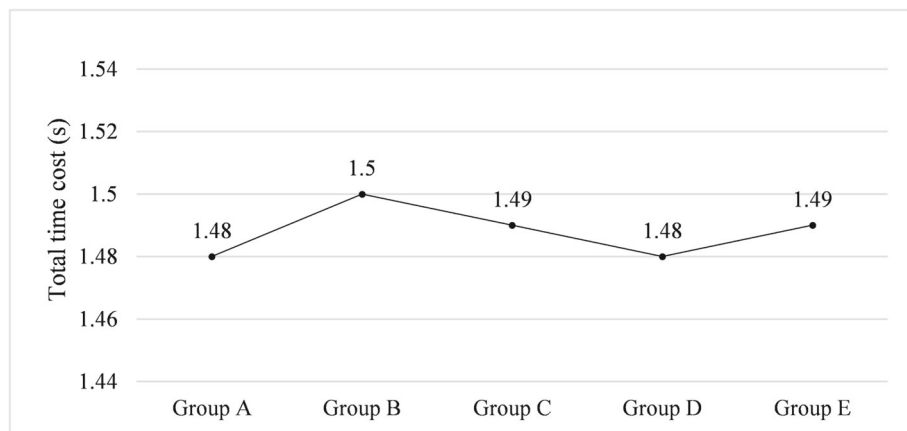


图12 整个翻译过程的总时间消耗

Table 6 Accuracy, precision and time cost between template matching and model

Query Types	Accuracy 1 ¹ (%)	Accuracy 2 ² (%)	Precision 1 ¹ (%)	Precision 2 ² (%)	Time Cost 1 ¹ (s)	Time Cost 2 ²
Time Point Query	96.7	96.7	86.7	86.7	1.32	1.26
Time Interval Query	96.7	96.7	86.7	86.7	1.33	1.36
Spatial Query	100	90	86.7	86.7	1.39	1.28
Spatio-temporal Query	98.3	88.3	88.3	88.3	1.4	1.36
Nearest Neighbor Query	100	86.7	96.7	88.3	1.64	1.41
Trajectory Similarity Query	96.7	85	88.3	76.7	1.68	1.49
Join Query	98.3	86.7	86.7	80	1.67	1.47

¹Using the trained model to determine the query type²Using template matching to determine the query type

2 2 s o C e m T	6 2 1	6 3 1	8 2 1	6 3 1	1 4 1	9 4 1	7 4 1
2 3 l l s o C e m T	2 3 1	3 3 1	9 3 1	4 1 1	4 6 1	8 6 1	7 6 1
2 n o s c e P	7 6 8	7 6 8	7 6 8	3 8 8	3 8 8	7 6 7	0 8 8
2 n o s c e P	7 6 8	7 6 8	7 6 8	3 8 8	7 6 9	3 8 8	7 6 8
2 y c a u c A	7 6 9	7 6 9	0 9 8	3 8 8	7 6 8	7 5 8	7 6 8
2 y c a u c A	7 6 9	7 6 9	0 1 9	3 9 9	0 1 9	7 9 9	3 8 9

e
p
y
y
e
u
q
e
h
e
n
m
e
e
d
o
i
e
d
o
m
d
e
n
a
e
h
g
h
s
U
2

6.3.4 User interaction

We implement an operator named *nlmo* for the user, and the user can use this operator to translate the corresponding natural language query. This operator takes natural language query as input, and the output of the system has three kinds if the input format is correct. The first is that the structured query statement cannot be constructed and the system directly ends the process; the second is that the system can provide query results for the user but does not meet the user's expectations. The third is the system can accurately give the query results expected by the user. The specific user-system interaction structure is shown in Fig. 13.

In the first case, the system will inform the user of the problem. Generally, there are two types of feedback information: (i) time, location, query object, query relation and other entities which are incorrect or non-existent; (ii) grammatical error and the query type that cannot be determined.

For the first type of feedback information, the entity queried by the user may be not exist, or the entity may be too vague. If the location information is missing, the system allows the user to give feedback to the administrator, and the administrator can add the corresponding location information, such as some personalized POI information that does not exist in the location knowledge base. For the second type of feedback information, the user can only refer to the query example, and then readjust the query to re-query. In the second case, if the user is dissatisfied with the query result, the user can ask the system to regenerate the query result. Three sets of possible query results will be generated inside the system and sorted, and the result with the highest score will be fed back to the user. If the user is not satisfied with the result, the system will give the result with the second score to the user, until all possible results are given. If the user is still dissatisfied with the result, the user has to adjust the query and re-query. If the user selects the result that does not get the highest score, then the result will be re-adjusted as the highest score.

NLIDBs are designed for users who use natural language to query, and we should conduct real user tests. We selected four volunteers for the test, two of them are master students who only understand common natural language queries and do not have any SQL knowledge. The other two are master students studying moving objects databases. They are not only familiar with the way natural language queries are proposed, but also the structured queries of moving objects, and can independently construct structured query sentences for database. Four volunteers need to participate in three tests separately: (i) The user uses the provided query to test and give feedback on the query; (ii) Based on the query used in the previous test, the user independently proposes a new query and gives feedback; (iii) The user interactively revises the query made by himself and gives feedback again. Specifically, in the first test, a total of 120 queries based on different types of queries will be given, that is, 30 queries of

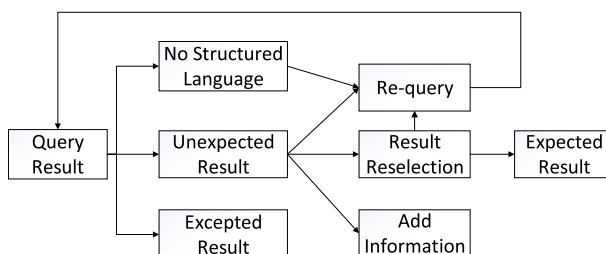


Fig. 13 User-system interaction structure

6.3.4 用户互动

我们为用户实现了一个名为 *nlmo* 的操作符，用户可以使用该操作符来翻译相应的自然语言查询。该操作符以自然语言查询作为输入，如果输入格式正确，系统的输出有三种情况。第一种是无法构建结构化查询语句，系统直接结束处理；第二种是系统可以为用户提供查询结果，但不符合用户的期望；第三种是系统可以准确地给出用户期望的查询结果。具体的用户-系统交互结构如图 13 所示。

在第一种情况下，系统会通知用户问题。一般来说，有两种类型的反馈信息：(i) 时间、地点、查询对象、查询关系和其他不正确或不存在的实体；(ii) 语法错误以及无法确定的查询类型。

对于第一类反馈信息，用户查询的实体可能不存在，或者该实体可能过于模糊。如果位置信息缺失，系统允许用户向管理员反馈，管理员可以添加相应的位置信息，例如一些在位置知识库中不存在的个性化POI信息。对于第二类反馈信息，用户只能参考查询示例，然后重新调整查询以再次查询。在第二种情况下，如果用户对查询结果不满意，用户可以要求系统重新生成查询结果。系统内部将生成三组可能的查询结果并进行排序，得分最高的结果将反馈给用户。如果用户对结果仍不满意，系统将向用户提供得分第二的结果，直到提供所有可能的结果为止。如果用户仍然不满意

NLIDBs 是为使用自然语言进行查询的用户设计的，我们应该进行实际用户测试。我们选择了四名志愿者进行测试，其中两名是只理解常见自然语言查询但没有任何 SQL 知识的硕士生。另外两名是研究移动对象数据库的硕士生。他们不仅熟悉自然语言查询的提出方式，还熟悉移动对象的结构化查询，并且能够独立构建数据库的结构化查询语句。四名志愿者需要分别参加三次测试：(i) 用户使用提供的查询进行测试并对查询提供反馈；(ii) 基于前一次测试中使用的查询，用户独立提出新的查询并提供反馈；(iii) 用户交互式修改自己提出的查询并再次提供反馈。具体来说，在第一次测试中，将提供基于不同类型查询的总共 120 条查询，也就是 30 条 qu

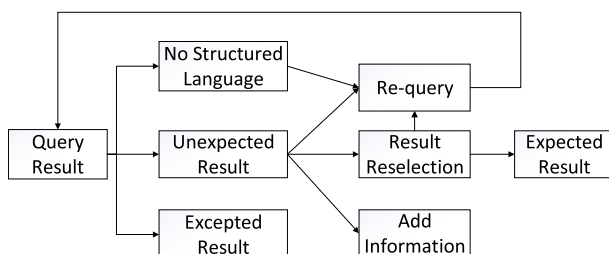


图13 用户-系统交互结构

each type for users to test. These queries can get structured sentences. Ordinary volunteers only need to give a judgment whether each query meets the expectations, and the expert volunteers not only need to give the judgment of the query but also give the reason for the error. In the second test, four volunteers are required to give queries that are different from the test examples. About 20 queries of each type are given, for a total of 80 queries, and we perform corresponding query tests. Similarly, ordinary volunteers need to give a judgment whether each query meets the expectations, and the expert volunteers not only need to give the judgment of the query but also give the reason for the error. In the last test, according to the query that users cannot get the expected result in the first and second tests, users need to interact with the system. Users can add certain additional entities to the database or let the system regenerate a new query. In the end, the volunteers give feedback on satisfaction of using the system. In addition, each volunteer is tested individually in an independent space, and each volunteer cannot communicate before completing all the tests, ensuring that the volunteers will not affect each other. Volunteers researching moving objects databases may construct high-quality and robust moving objects queries due to their inherent knowledge, and the queries may be quite different from those constructed by ordinary volunteers. Therefore, ordinary volunteers and expert volunteers are selected for comparison, to avoid subjective differences due to inherent knowledge.

From the previously constructed moving objects corpus, we get 120 robust natural language queries for volunteers to test through the selection of experts. At the same time, the moving objects queries proposed by users in the second round of testing are also used as the test dataset. The two test datasets are slightly different. We specifically screen out the queries proposed by ordinary volunteers and the queries extracted from the papers and journals as examples for comparison. The queries proposed by users and the given queries are basically the same in the natural language query construction of different moving objects. However, the queries proposed by volunteers, especially ordinary volunteers, are slightly different from the expected queries supported by the system. For example, the query relation and query object in the sample query are very clear and exist in the dataset, but sometimes the query object and query relation given by ordinary volunteers are rather ambiguous, as shown in Table 7.

The experiment conduct a separate analysis for each round of testing. We show the results of each round of testing. In the first round of testing, both ordinary volunteers and expert volunteers give great satisfaction, and each category has great satisfaction. The satisfaction of ordinary volunteers and expert volunteers are 92.5% and 98.6% respectively, which are almost

Table 7 Query comparison

Sample Query	Volunteer Query
Where did the train 123 go at 8am?	Where did the train 24 go from 8am to 9am?
Find the trains that pass the place Tiergarten from 8:50 to 11:30.	Which buses went to the Suzhou station from 8:05 to 10:00 o'clock?
Tell me the nearest neighbor to the train 1 between 9:00 and 9:01 o'clock.	Could you find the 12 nearest neighbors of the green train during [2:45, 3:56]?
Would you mind telling me the trajectory that is similar to the fastest train 134 this afternoon from 7pm to 9pm?	Can you show me two trains that are similar to the train 3 that passes People's Square?
Please list the common stations that train 12 and Train 67 pass through from 8:00am to 10:05am.	Could you help me list all the same kindergartens that bus 23 and bus 22 passed between 9 am and 11 am on March 10th?

每种类型供用户测试。这些查询可以生成结构化句子。普通志愿者只需判断每个查询是否符合预期，而专家志愿者不仅需要对查询作出判断，还需给出错误的原因。在第二次测试中，需要四名志愿者给出与测试示例不同的查询。每种类型约给出 20 个查询，总计 80 个查询，并进行相应的查询测试。同样，普通志愿者需要判断每个查询是否符合预期，而专家志愿者不仅需要对查询作出判断，还需给出错误的原因。在最后一次测试中，根据用户在第一次和第二次测试中无法获得期望结果的查询，用户需要与系统进行交互。用户可以向数据库添加某些附加实体或让系统重新生成新的查询。最终，志愿者提供反馈

从之前构建的移动目标语料库中，我们通过专家的筛选得到了120条稳健的自然语言查询供志愿者测试。同时，用户在第二轮测试中提出的移动目标查询也被用作测试数据集。两个测试数据集略有不同。我们专门筛选出普通志愿者提出的查询以及从论文和期刊中提取的查询作为对比示例。用户提出的查询与给定查询在不同移动目标的自然语言查询构建上基本相同。然而，志愿者提出的查询，尤其是普通志愿者提出的查询，与系统所支持的期望查询略有不同。例如，样本查询中的查询关系和查询对象非常清晰，并且存在于数据集中，但有时普通志愿者给出的查询对象和查询关系相当模糊，如所

实验对每一轮测试进行单独分析。我们展示每一轮测试的结果。在第一轮测试中，普通志愿者和专家志愿者都给予了很高的满意度，每个类别的满意度都很高。普通志愿者和专家志愿者的满意度分别为92.5%和98.6%，几乎

表7 查询比较

Sample Query	Volunteer Query
Where did the train 123 go at 8am?	Where did the train 24 go from 8am to 9am?
Find the trains that pass the place Tiergarten from 8:50 to 11:30.	Which buses went to the Suzhou station from 8:05 to 10:00 o'clock?
Tell me the nearest neighbor to the train 1 between 9:00 and 9:01 o'clock.	Could you find the 12 nearest neighbors of the green train during [2:45, 3:56]?
Would you mind telling me the trajectory that is similar to the fastest train 134 this afternoon from 7pm to 9pm?	Can you show me two trains that are similar to the train 3 that passes People's Square?
Please list the common stations that train 12 and Train 67 pass through from 8:00am to 10:05am.	Could you help me list all the same kindergartens that bus 23 and bus 22 passed between 9 am and 11 am on March 10th?

the same, because these test cases are extracted from high-quality journals and conference papers. Besides, the test cases are constructed through professional modification and the results are within expectations. Among the queries that do not meet users' expectations, expert volunteers give corresponding reasons. For example, the query object does not exist or the database does not support a certain type of query object. The reasons of the errors given by the expert volunteers are reasonable. The second round of testing is more challenging. We need to pay attention to whether the inherent knowledge of the expert will affect the construction of the query, but the results show that there is no big gap between ordinary volunteers and expert volunteers. The main reason is that the first round of testing gives ordinary volunteers a certain understanding of the query, and they will not propose vague queries. However, ordinary volunteers do not know the query types of moving objects, and the query types are not evenly distributed. The distribution of the queries proposed by ordinary volunteers and expert volunteers is shown in the following table. Similarly, several volunteers test their queries and give corresponding feedback. The queries proposed by the expert volunteers are relatively standardized and can be well translated into structured languages. Two of the queries do not get the expected results because the query relation does not exist. However, three queries proposed by ordinary volunteers show vague semantics, the system can not construct structured sentences, and the three queries do not meet the expectations of volunteers. In the third round of testing, volunteers can improve the unsatisfactory queries in the first and second rounds. If the reason is POI and other locations do not exist, every volunteer can make suggestions to expand the location knowledge base. Volunteers who do not get expected results can ask the system to regenerate the results. In the end, all volunteers give reasons for unsatisfactory results so that we can optimize the system. The specific results are shown in Table 8. One query proposed by ordinary volunteers does not meet expectations, because the system judges the nearest neighbor query as a trajectory similarity query, and the volunteers can not get the expected results even if they ask the system to regenerate the results. For queries with missing locations, users can get the expected results after adding locations.

7 System development

We design and develop NALMO, a natural language query translation system for moving objects. The system is integrated in the scalable database system SECONDO. The final form is an algebra module in SECONDO, and the algebra implements an operator named *nlmo*. Users can use this operator to translate the corresponding natural language query on the database in the SECONDO system, and then execute the translated executable query to obtain the query result.

Table 8 Volunteer Satisfaction

Volunteer	First Round Satisfaction	Second Round Satisfaction	Third Round Satisfaction
Ordinary Volunteer1	115/120 (95.8%)	72/80 (90%)	12/13 (92.3%)
Ordinary Volunteer2	116/120 (96.7%)	70/80 (87.5%)	13/14 (92.9%)
Expert Volunteer1	118/120 (98.3%)	78/80 (97.5%)	4/4 (100%)
Expert Volunteer2	119/120 (99.2%)	77/80 (96.3%)	4/4 (100%)

相同，因为这些测试案例是从高质量的期刊和会议论文中提取的。此外，这些测试案例是通过专业修改构建的，结果在预期范围内。对于不符合用户期望的查询，专家志愿者会给出相应的理由。例如，查询对象不存在或数据库不支持某种类型的查询对象。专家志愿者给出的错误原因是合理的。第二轮测试更具挑战性。我们需要关注专家的固有知识是否会影响查询的构建，但结果显示普通志愿者和专家志愿者之间没有很大差距。主要原因是第一轮测试让普通志愿者对查询有了一定的理解，他们不会提出模糊的查询。然而，普通志愿者不了解移动对象的查询类型，并且q

7 系统开发

我们设计并开发了 NALMO，一个用于移动对象的自然语言查询翻译系统。该系统已集成在可扩展的数据库系统 SECONDO 中。最终形式是 SECONDO 中的一个代数模块，该代数实现了一个名为 *nlmo* 的操作符。用户可以使用此操作符将 SECONDO 系统中数据库的相应自然语言查询进行翻译，然后执行翻译后的可执行查询以获得查询结果。

表 8 志愿者满意度

Volunteer	First Round Satisfaction	Second Round Satisfaction	Third Round Satisfaction
Ordinary Volunteer1	115/120 (95.8%)	72/80 (90%)	12/13 (92.3%)
Ordinary Volunteer2	116/120 (96.7%)	70/80 (87.5%)	13/14 (92.9%)
Expert Volunteer1	118/120 (98.3%)	78/80 (97.5%)	4/4 (100%)
Expert Volunteer2	119/120 (99.2%)	77/80 (96.3%)	4/4 (100%)

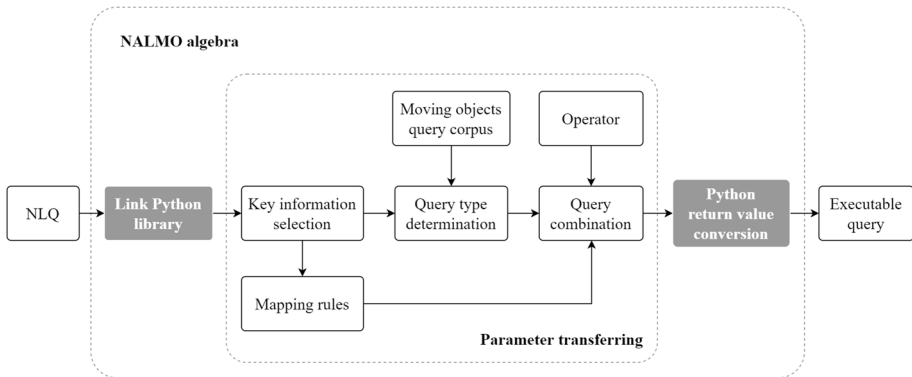


Fig. 14 The system development architecture

There are three main steps of system development: (i) use Python to implement the functions of the semantic understanding part and the query translation part; (ii) create an algebra that implements the functions of NALMO system; (iii) activate and compile the algebra for normal use of the operator *nlmo*. The architecture of the system development is shown in Fig. 14. In this process, query processing in natural language understanding is programmed in Python, while structured statement generation is implemented based on SECONDO and programmed in C++. Therefore, the main challenge in system development is the processing from Python modules to C++ modules. The final solution is to use the C++ library provided by Python to call the semantic understanding and query translation modules in the C++ program, mainly including (i) *the linking of the Python library*, (ii) *the transfer of parameters* and (iii) *the conversion of Python return values*.

The linking of the Python library Header files and library files are already included in the Python installation directory. We need to link to this library before using the Python module. We can call the Python statement directly, and then we can use the module functions in the C++ library provided by Python for initialization. Finally, we can use the corresponding functions to directly run Python statements or call the required functions. After encapsulating the natural language understanding module for moving objects query into a C++ module, we can use C++ to run the natural language understanding script file and call the functions.

The transfer of parameters We need to use C++ to pass parameters to Python modules, that is, pass parameters when calling functions in Python modules. The arguments of Python are actually tuples, and passing an argument is actually constructing a suitable tuple.

The conversion of Python return values We need to switch the working directory of Python to the path where the module is located, and load the module by the module name instead of the file name. Module loading or function loading needs to be verified successfully, otherwise the process may cause a stack error and program crashing. We should use suitable functions to dereference the object for Python garbage collection.

8 Conclusion

We design and develop a natural language interface for moving objects databases. The proposal supports translating five types of queries. Experiment results demonstrate the advantage

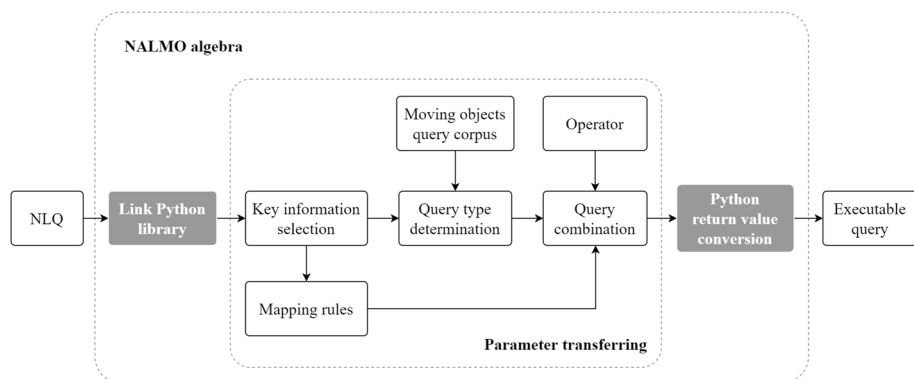


图14 系统开发架构

系统开发有三个主要步骤：（i）使用 Python 实现语义理解部分和查询翻译部分的功能；（ii）创建一个实现 NALMO 系统功能的代数；（iii）激活并编译该代数以便 *nlmo* 操作符的正常使用。系统开发的架构如图 14 所示。在此过程中，自然语言理解中的查询处理在 Python 中编程，而结构化语句生成基于 SECONDO 实现，并在 C++ 中编程。因此，系统开发的主要挑战是从 Python 模块到 C++ 模块的处理。最终的解决方案是使用 Python 提供的 C++ 库来调用 C++ 程序中的语义理解和查询翻译模块，主要包括（i）*the linking of the Python library*，（ii）*the transfer of parameters* 和（iii）*the conversion of Python return values*。

Python 库的链接 头文件和库文件已经包含在 Python 安装目录中。我们需要在使用 Python 模块之前链接到该库。我们可以直接调用 Python 语句，然后可以使用 Python 提供的 C++ 库中的模块函数进行初始化。最后，我们可以使用相应的函数直接运行 Python 语句或调用所需的函数。在将用于移动对象查询的自然语言理解模块封装到 C++ 模块后，我们可以使用 C++ 运行自然语言理解脚本文件并调用函数。

参数传递 我们需要使用 C++ 将参数传递给 Python 模块，也就是在调用 Python 模块中的函数时传递参数。Python 的参数实际上是元组，传递一个参数实际上是构造一个合适的元组。

Python 返回值的转换 我们需要将 Python 的工作目录切换到模块所在的路径，并通过模块名而不是文件名加载模块。模块加载或函数加载需要成功验证，否则该过程可能导致堆栈错误和程序崩溃。我们应该使用合适的函数来取消引用对象以便 Python 垃圾回收。

8 结论

我们设计并开发了一个面向移动对象数据库的自然语言接口。该方案支持翻译五种类型的查询。实验结果显示了其优势。

of our method in terms of accuracy, precision, efficiency and usability. An important task in the future is to further improve the fault tolerance and the association prompts of input errors. When the input is not successfully sent into the system, NALMO is able to determine whether the user has entered similar information before. Considering that we can use the index to improve moving objects query efficiency, the corresponding index structure needs to be introduced in the future. The moving objects database has many index structures, such as R-Tree, TPR-Tree, SETI and other indexes. Supporting these indexes during the construction of structured language is a challenge task.

Acknowledgements This work is supported by National Natural Science Foundation of China (61972198), Natural Science Foundation of Jiangsu Province of China (BK20191273).

Author Contributions Conflicts of reviewers: Ralf Hartmut Güting, rhg@fernuni-hagen.de is the Ph.D advisor of Author Jianqiu Xu. Hua Lu, luhua@ruc.dk, Roskilde University is the co-author of the paper. Zhifeng Bao, zhifeng.bao@rmit.edu.au, RMIT University has the co-authorship with Jianqiu Xu. Ouri E. Wolfson, wolfson@uic.edu, University of Illinois at Chicago has the co-authorship with Jianqiu Xu. Yu Zheng, msyuzheng@outlook.com, JD Finance has the co-authorship with Jianqiu Xu. Bin Yao yaobin@cs.sjtu.edu.cn, Shanghai Jiao Tong University has the co-authorship with Jianqiu Xu.

Data Availability The datasets generated and analysed during this study are available at: <https://pan.nuaa.edu.cn/share/3208e9a270de8389f9477d1cda> or <https://pan.baidu.com/s/1TVTPPZmDQ4nT8E7wXOXaAA?pwd=3qjc>.

Appendix

Table 9 Moving objects queries in natural language

Id	Query Expression
Q1	Which places did the trains go between 9:00 and 10:00 o'clock?
Q2	Which trains pass the station Mehringdamm from 8:00 to 10:00?
Q3	Please tell me the position of train 3 between 11am and 1pm?
Q4	Who can tell me which train has the similar trajectory of the train 134 at 7am?
Q5	Which trains are the 5 continuous nearest neighbors of the train 345 from 8:00 to 9:00 o'clock?
Q6	Did any trains pass Westhafen between 1pm and 1:30pm?
Q7	Find the trains that pass Mehringdamm from 8:20 to 10:59.
Q8	Which train is the nearest neighbor of the train 123 from 9am to 10am?
Q9	Which train is the nearest neighbor to the train 345 between [9am, 10am]?
Q10	Could you tell me which train is similar to the train 77 during [5pm, 9pm]?
Q11	What is the location of the train 1 from 8:00 to 9:10?
Q12	Please find almost 20 continuous nearest neighbors of the train 28 which is in Nanjing between 2am and 4am.
Q13	Which trains arrive at the Zoogarten between 8:18 and 10:27?
Q14	May I know the similar trajectory of the taxi 2 from 1:13 to 2:24?
Q15	What is the location of train 44 at 23:01?
Q16	Do you know the route of the train 56 between 7:00am and 16:00pm?
Q17	Query the trains that have left from 11am to 2pm.
Q18	Find the periods that the train 11 went to Alexanderplatz.

从准确性、精确性、效率和可用性方面对我们的方法进行了评估。未来的一项重要任务是进一步提高容错能力和输入错误的关联提示。当输入未能成功发送到系统时，NALMO能够判断用户之前是否输入过类似信息。考虑到我们可以使用索引提高移动对象查询效率，未来需要引入相应的索引结构。移动对象数据库有许多索引结构，例如R-Tree、TPR-Tree、SETI等索引。在构建结构化语言时支持这些索引是一项具有挑战性的任务。

致谢 本工作得到中国国家自然科学基金（61972198）、江苏省自然科学基金（BK20191273）的资助。

作者贡献 评审者冲突：Ralf Hartmut Güting, rhg@fernuni-hagen.de 是作者许建秋的博客导师。卢华, luhua@ruc.dk, 哥本哈根大学（Roskilde University）是本文的共同作者。包志峰, zhifeng.bao@rmit.edu.au, RMIT大学与许建秋有共同作者关系。Ouri E. Wolfson, wolfson@uic.edu, 芝加哥伊利诺伊大学与许建秋有共同作者关系。郑宇, msyuzheng@outlook.com, 京东金融与许建秋有共同作者关系。姚斌, yaobin@cs.sjtu.edu.cn, 上海交通大学与许建秋有共同作者关系。

数据可用性 本研究生成和分析的数据集可在以下网址获取：<https://pan.nuaa.edu.cn/share/3208e9a270de8389f9477d1cda> 或 <https://pan.baidu.com/s/1TVTPPZmDQ4nT8E7wXOXaAA?pwd=3qjc>。

附录

表9 自然语言中的移动对象查询

Id	Query Expression
Q1	Which places did the trains go between 9:00 and 10:00 o'clock?
Q2	Which trains pass the station Mehringdamm from 8:00 to 10:00?
Q3	Please tell me the position of train 3 between 11am and 1pm?
Q4	Who can tell me which train has the similar trajectory of the train 134 at 7am?
Q5	Which trains are the 5 continuous nearest neighbors of the train 345 from 8:00 to 9:00 o'clock?
Q6	Did any trains pass Westhafen between 1pm and 1:30pm?
Q7	Find the trains that pass Mehringdamm from 8:20 to 10:59.
Q8	Which train is the nearest neighbor of the train 123 from 9am to 10am?
Q9	Which train is the nearest neighbor to the train 345 between [9am, 10am]?
Q10	Could you tell me which train is similar to the train 77 during [5pm, 9pm]?
Q11	What is the location of the train 1 from 8:00 to 9:10?
Q12	Please find almost 20 continuous nearest neighbors of the train 28 which is in Nanjing between 2am and 4am.
Q13	Which trains arrive at the Zoogarten between 8:18 and 10:27?
Q14	May I know the similar trajectory of the taxi 2 from 1:13 to 2:24?
Q15	What is the location of train 44 at 23:01?
Q16	Do you know the route of the train 56 between 7:00am and 16:00pm?
Q17	Query the trains that have left from 11am to 2pm.
Q18	Find the periods that the train 11 went to Alexanderplatz.

Table 9 continued

Id	Query Expression
Q19	Did any trains pass the station Alexanderplatz during [11am, 12am] today?
Q20	Please tell me which trains have been to the Mehringdamm station from 8:20 to 10:00?
Q21	Find precisely seven closest neighbors of the train 8 during [10am, 6pm].
Q22	Which train has the similar trajectory of the train 17 between 17:00 and 19:00 o'clock?
Q23	Would you mind telling me the trajectory that is close to the fastest train 29 at 18:19 in Beijing?
Q24	May I know which train is similar to the train 1 that passes Sydney Opera House near me?
Q25	Tell me which train has the similar trajectory of the train 37 at Shanghai station during [7:19, 11:19].
Q26	Please tell me which trains have been to the Koepenick from 8:20 to 10:00?
Q27	Which train is the nearest neighbor to the train 345 between [9am, 10am]?
Q28	Where was the train 5 at 17:00 o'clock on Saturday?
Q29	Find the periods that train 17 passed by the Arch of Triumph?
Q30	From 8am to 9am, which trains are five nearest neighbors to the train 47?
Q31	Among these trains, please list at least three trains that can reach Beijing from 9:00 to 9:20.
Q32	Find the same driving path for train1 and train23 from 8:00 to 14:00.
Q33	Show the same path of train2 and train4 between 6:40am and 10:15am.
Q34	Please help me determine the common trajectory of train 13 and train 2 between 5pm and 7pm that pass Beijing Station.
Q35	Can you help me find out exactly what the same trajectory of the three trains from 10:00am to 14:00pm?

References

- Affolter K, Stockinger K, Bernstein A (2019) A comparative survey of recent natural language interfaces for databases. *VLDB J* 28(5):793–819
- Amsterdamer Y, Kukliansky A, Milo T (2015) Nl2cm: A natural language interface to crowd mining. In: *ACM SIGMOD*, pp 1433–1438
- Artzi Y, Zettlemoyer LS (2011) Bootstrapping semantic parsers from conversations. *EMNLP* 2011:421–432
- Baik C, Jagadish HV, Li Y (2019) Bridging the semantic gap with SQL query logs in natural language interfaces to databases. *ICDE* 2019:374–385
- Booth J, Eugenio BD, Cruz IF, Wolfson O (2015) Robust natural language processing for urban trip planning. *Appl Artif Intell* 29(9):859–903
- Chakka VP, Everspauth A, Patel JM (2003) Indexing large trajectory data sets with SETI. In: *CIDR*
- Cheng J, Reddy S, Saraswat VA, Lapata M (2017) Learning structured natural language representations for semantic parsing. *ACL* 2017:44–55
- Chon HD, Agrawal D, Abadi AE (2003) Range and knn query processing for moving objects in grid model. *Mob Networks Appl* 8(4):401–412
- Damljanovic D, Agatonovic M, Cunningham H (2011) Freya: An interactive way of querying linked data using natural language. *ESWC, Lecture Notes in Computer Science* 7117:125–138
- Deutch D, Frost N, Gilad A, Haimovich T (2018) Nlprovenans: Natural language provenance for non-answers. *Proc VLDB Endow* 11(12):1986–1989
- Deutch D, Frost N, Gilad A (2020) Explaining natural language query results. *VLDB J* 29(1):485–508
- Dong L, Lapata M (2018) Coarse-to-fine decoding for neural semantic parsing. *ACL* 2018:731–742
- Ferré S (2017) Sparklis: An expressive query builder for SPARQL endpoints with guidance in natural language. *Semantic Web* 8(3):405–418
- Giordani A, Moschitti A (2012) Translating questions to SQL queries with generative parsers discriminatively reranked. In: *COLING*, pp 401–410

表9 续

Id	Query Expression
Q19	Did any trains pass the station Alexanderplatz during [11am, 12am] today?
Q20	Please tell me which trains have been to the Mehringdamm station from 8:20 to 10:00?
Q21	Find precisely seven closest neighbors of the train 8 during [10am, 6pm].
Q22	Which train has the similar trajectory of the train 17 between 17:00 and 19:00 o'clock?
Q23	Would you mind telling me the trajectory that is close to the fastest train 29 at 18:19 in Beijing?
Q24	May I know which train is similar to the train 1 that passes Sydney Opera House near me?
Q25	Tell me which train has the similar trajectory of the train 37 at Shanghai station during [7:19, 11:19].
Q26	Please tell me which trains have been to the Koepenick from 8:20 to 10:00?
Q27	Which train is the nearest neighbor to the train 345 between [9am, 10am]?
Q28	Where was the train 5 at 17:00 o'clock on Saturday?
Q29	Find the periods that train 17 passed by the Arch of Triumph?
Q30	From 8am to 9am, which trains are five nearest neighbors to the train 47?
Q31	Among these trains, please list at least three trains that can reach Beijing from 9:00 to 9:20.
Q32	Find the same driving path for train1 and train23 from 8:00 to 14:00.
Q33	Show the same path of train2 and train4 between 6:40am and 10:15am.
Q34	Please help me determine the common trajectory of train 13 and train 2 between 5pm and 7pm that pass Beijing Station.
Q35	Can you help me find out exactly what the same trajectory of the three trains from 10:00am to 14:00pm?

参考文献

1. Affolter K, Stockinger K, Bernstein A (2019) 最近数据库自然语言接口的比较调查. VLDB J 28(5): 793–819
2. Amsterdam Y, Kukliansky A, Milo T (2015) NI2cm: 一种面向众包挖掘的自然语言接口。在: ACM SIGMOD, 页 1433–1438
3. Artzi Y, Zettlemoyer LS (2011) 从对话中自举语义解析器. E MNLP 2011:421–432
4. Baik C, Jagadish HV, Li Y (2019) 利用 SQL 查询日志弥合数据库自然语言接口中的语义鸿沟. ICDE 2019:374–385
5. Booth J, Eugenio BD, Cruz IF, Wolfson O (2015) 面向城市出行规划的鲁棒自然语言处理. Appl Artif Intell 29(9):859–903
6. Chakka VP, Everspaugh A, Patel JM (2003) 使用 SETI 对大规模轨迹数据集进行索引。在: CIDR
7. Cheng J, Reddy S, Saraswat VA, Lapata M (2017) 学习结构化自然语言表示用于语义解析. ACL 2017:44–55
8. Chon HD, Agrawal D, Abbadi AE (2003) 网格中移动对象的范围和 k 近邻查询处理

15. Gur I, Yavuz S, Su Y, Yan X (2018) Dialsq: Dialogue based structured query generation. In: ACL, pp 1339–1349
16. Gü R, Almeida Vd, Ding Z (2006) Modeling and querying moving objects in networks. *Vldb Journal* 15(2):165–190
17. Güting RH, Schneider M (2005) *Moving Objects Databases*. Elsevier
18. Güting RH, Behr T, Düntgen C (2010) SECONDO: A platform for moving objects database research and for publishing and integrating research implementations. *IEEE Data Eng Bull* 33(2):56–63
19. Güting RH, Behr T, Xu J (2010) Efficient k-nearest neighbor search on moving object trajectories. *Vldb J* 19(5):687–714
20. Iyer S, Konstas I, Cheung A, Krishnamurthy J, Zettlemoyer L (2017) Learning a neural semantic parser from user feedback. In: ACL, pp 963–973
21. Jensen C, Lu H, Yang B (2009) Graph model based indoor tracking. In: MDM, pp 122–131
22. Jia Z, Abujabal A, Roy RS, Strötgen J, Weikum G (2018) TEQUILA: temporal question answering over knowledge bases. In: ACM CIKM, pp 1807–1810
23. Jia Z, Pramanik S, Roy RS, Weikum G (2021) Complex temporal question answering on knowledge graphs. In: CIKM '21, pp 792–802
24. Kamath A, Das R (2019) A survey on semantic parsing. In: AKBC
25. Kim J, Hong D, Kang H, Han K (2006) TMOM: A moving object main memory-based DBMS for telematics services. In: W2GIS 2006, Lecture Notes in Computer Science, vol 4295, pp 259–268
26. Klein D, Manning CD (2003) Accurate unlexicalized parsing. In: ACL, pp 423–430
27. Li F, Jagadish HV (2014) Constructing an interactive natural language interface for relational databases. *Proc VLDB Endow* 8(1):73–84
28. Li J, Wang W, Ku W, Tian Y, Wang H (2019) Spatialnli: A spatial domain natural language interface to databases using spatial comprehension. In: ACM SIGSPATIAL, pp 339–348
29. Li Y, Rafiei D (2017) Natural language data management and interfaces: Recent development and open challenges. *SIGMOD 2017*:1765–1770
30. Li Y, Rafiei D (2018) *Natural Language Data Management and Interfaces*. Synthesis Lectures on Data Management 10(2):1–156
31. Li Y, Yang H, Jagadish HV (2007) Nalix: A generic natural language search environment for XML data. *ACM Trans Database Syst* 32(4):30
32. Liu P, Wang M, Cui J, Li H (2021) Top-k competitive location selection over moving objects. *Data Sci Eng* 6(4):392–401
33. Mikolov T, Sutskever I, Chen K, Corrado GS, Dean J (2013) Distributed representations of words and phrases and their compositionality. In: NIPS, pp 3111–3119
34. Miller GA (1995) Wordnet: A lexical database for english. *Commun ACM* 38(11):39–41
35. Mokbel MF, Sakr MA, Xiong L, Züfle A et al (2022) Mobility data science (dagstuhl seminar 22021). *Dagstuhl Reports* 12(1):1–34
36. Pasupat P, Liang P (2015) Compositional semantic parsing on semi-structured tables. *ACL 2015*:1470–1480
37. Pelekis N, Frentzos E, Giatrakos N, Theodoridis Y (2008) HERMES: aggregative LBS via a trajectory DB engine. pp 1255–1258
38. Pfoser D, Jensen CS, Theodoridis Y (2000) Novel approaches in query processing for moving object trajectories. In: VLDB, pp 395–406
39. Poon H, Domingos PM (2009) Unsupervised semantic parsing. In: EMNLP 2009:1–10
40. Popescu A, Etzioni O, Kautz HA (2003) Towards a theory of natural language interfaces to databases. In: IUI, pp 149–157
41. Popescu A, Armanasu A, Etzioni O, Ko D, Yates A (2004) Modern natural language interfaces to databases: Composing statistical parsing with semantic tractability. In: COLING, pp 141–147
42. Price PJ (1990) Evaluation of spoken language systems: the ATIS domain. In: *Speech and natural language*, pp 91–95
43. Saha D, Floratou A, Sankaranarayanan K, Minhas UF, Mittal AR, Özcan F (2016) ATHENA: an ontology-driven system for natural language querying over relational data stores. *Proc VLDB Endow* 9(12):1209–1220
44. Stratica N, Kosseim L, Desai BC (2005) Using semantic templates for a natural language interface to the CINDI virtual library. *Data Knowl Eng* 55(1):4–19
45. Suhr A, Iyer S, Artzi Y (2018) Learning to map context-dependent sentences to executable formal queries. In: NAACL-HLT, pp 2238–2249
46. Tang LR, Mooney RJ (2001) Using multiple clause constructors in inductive logic programming for semantic parsing. *EMCL, Lecture Notes in Computer Science* 2167:466–477

15. Gur I, Yavuz S, Su Y, Yan X (2018) Dialsq1: 基于对话的结构化查询生成。载于: ACL, 第1339–1349页
16. Gü R, Almeida Vd, Ding Z (2006) 网络中移动对象的建模与查询。《VLDB期刊》15(2): 165–190
17. Güting RH, Schneider M (2005) 移动对象数据库。Elsevier
18. Güting RH, Behr T, Düntgen C (2010) SECONDO: 用于移动对象数据库研究以及发布和整合研究实现的平台。《IEEE数据工程通报》33(2): 56–63
19. Güting RH, Behr T, Xu J (2010) 移动对象轨迹上的高效k近邻搜索。《VLDB期刊》19(5): 687–714
20. Iyer S, Konstas I, Cheung A, Krishnamurthy J, Zettlemoyer L (2017) 从用户反馈中学习神经语义解析器。载于: ACL, 第963–973页
21. Jensen C, Lu H, Yang B (2009) 基于图模型的室内跟踪。载于: MDM, 第122–131页
22. Jia Z, Abujabal A, Roy RS, Strötgen J, Weikum G (2018) TEQUILA: 基于知识库的时间性问答。载于: ACM CIKM, 第1807–1810页
23. Jia Z, Pra

47. Tao Y, Papadias D (2001) Mv3r-tree: A spatio-temporal access method for timestamp and interval queries. In: VLDB, pp 431–440
48. Ulm G, Smith S, Nilsson A, Gustavsson E, Jirstrand M (2021) OODIDA: on-board/off-board distributed real-time data analytics for connected vehicles. *Data Sci Eng* 6(1):102–117
49. Vazirgiannis M, Theodoridis Y, Sellis TK (1998) Spatio-temporal composition and indexing for large multimedia applications. *Multim Syst* 6(4):284–298
50. Wang X, Xu J, Wang Y (2020) NLMO: towards a natural language tool for querying moving objects. *MDM* 2020:228–229
51. Wang X, Xu J, Lu H (2021) NALMO: A natural language interface for moving objects databases. *SSTD* 2021:1–11
52. Wang Y, Berant J, Liang P (2015) Building a semantic parser overnight. *ACL* 2015:1332–1342
53. Wong YW, Mooney RJ (2006) Learning for semantic parsing with statistical machine translation. In: *HLT-NAACL*, pp 439–446
54. Xiong X, Elmongui HG, Chai X, Aref WG (2007) Place: A distributed spatio-temporal data stream management system for moving objects. *MDM* 2007:44–51
55. Xu J, Güting RH (2013) A generic data model for moving objects. *GeoInformatica* 17(1):125–172
56. Yaghmazadeh N, Wang Y, Dillig I, Dillig T (2017) Sqlizer: query synthesis from natural language. *Proc ACM Program Lang* 1(OOPSLA):63:1–63:26
57. Yin P, Neubig G (2017) A syntactic neural model for general-purpose code generation. *ACL* 2017:440–450
58. Yuan H, Li G (2021) A survey of traffic prediction: from spatio-temporal data to intelligent transportation. *Data Sci Eng* 6(1):63–85
59. Zenz G, Zhou X, Minack E, Siberski W, Nejd W (2009) From keywords to semantic queries - incremental query construction on the semantic web. *J Web Semant* 7(3):166–176
60. Zheng W, Cheng H, Zou L, Yu JX, Zhao K (2017) Natural language question/answering: Let users talk with the knowledge graph. *CIKM* 2017:217–226
61. Zheng Z, Li F, Huang M, Zhu X (2010) Learning to link entities with knowledge base. In: *HLT-NAACL*, pp 483–491
62. Sen J, Lei C, Quamar A, Ozcan F, Efthymiou V, Dalmia A, Stager G, Mittal AR, Saha D, Sankaranarayanan K (2020) ATHENA++: natural language querying for complex nested SQL queries. *Proc VLDB Endow* 13(11):2747–2759

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

47. Tao Y, Papadias D (2001) Mv3r-tree: 一种用于时间戳和区间查询的时空访问方法。发表于: VLDB, 第431–440页
48. Ulm G, Smith S, Nilsson A, Gustavsson E, Jirstrand M (2021) OODIDA: 面向联网车辆的车载/车外分布式实时数据分析。数据科学与工程 6(1):102–117
49. Vazirgiannis M, Theodoridis Y, Sellis TK (1998) 面向大型多媒体应用的时空构成与索引。多媒体系统 6(4):284–298
50. Wang X, Xu J, Wang Y (2020) NLMO: 面向移动对象查询的自然语言工具。MDM 2020:228–229
51. Wang X, Xu J, Lu H (2021) NALMO: 移动对象数据库的自然语言接口。SSTD 2021:1–11
52. Wang Y, Berant J, Liang P (2015) 一夜之间构建语义解析器。ACL 2015:1332–1342
53. Wong YW, Mooney RJ (2006) 使用统计机器翻译进行语义解析学习。发表于: HLT-NAACL, 第439–446页
54. Xiong X, Elmongui HG, Chai X, Aref WG (2007) PLACE: 一种分布式时空数据流

出版者声明 Springer Nature 对已发表地图和机构隶属关系中的管辖权主张保持中立。

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.